



National University of Computer and Emerging Sciences, Lahore



Manzil: Intelligent Traffic & Emergency Routing

Muhammad Ali Mahoon	23L-0531	BS(CS)
Muzammil Ahmed Bhatti	23L-0688	BS(CS)
Mohammad Ibrahim Salman	23L-0723	BS(CS)

Supervisor: Miss Umm-e-Ammarah

Final Year Project

10 October 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: Manzil: Intelligent Traffic & Emergency Routing

is the sole contribution of the author(s), and no part thereof has been reproduced in a form that can be considered plagiarism. All referenced material has been used to support the argument and cited properly. We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: 10 October 2026

Name: Muhammad Ali Mahoon

Signature: 

Name: Muzammil Ahmed Bhatti

Signature: 

Name: Mohammad Ibrahim Salman

Signature: 

Authors' Declaration

This states the Authors' declaration that the work presented in the report is their own and has not been submitted or presented previously to any other institution or organization.

Abstract

Manzil is a simulation-based research and development prototype for adaptive urban traffic control and emergency routing. It combines video-based vehicle detection and tracking, traffic-state estimation, comparative signal-control policies, congestion-aware ambulance routing, and a moving green corridor within a SUMO test environment. A React dashboard and FastAPI services are proposed to expose system state and experimental records. This report specifies the requirements, research method, architecture, and controlled evaluation protocol; implementation and measured results remain subsequent project work.

Executive Summary

Traffic congestion affects daily movement, while an avoidable delay to an ambulance can have more serious consequences. Conventional signal plans are prepared for expected demand and cannot respond to every short-term imbalance. Distance-only route guidance can also direct an emergency vehicle toward a short but heavily queued road. Manzil addresses these related problems through a controlled research prototype rather than a public-road deployment. It will examine whether observable traffic conditions can support responsive signal decisions, whether ambulance routes can account for current network delay, and whether selected intersections can provide temporary priority without imposing unreasonable disruption on ordinary traffic. For a supervisor, evaluator, or prospective public-sector stakeholder, the practical value is a repeatable environment in which alternative policies can be compared before investment in roadside equipment or operational integration is considered. The project does not promise citywide improvement or certified emergency operation. It provides a disciplined way to determine which parts of the approach are beneficial, what costs accompany those benefits, and which technical or institutional limitations would have to be addressed before field-oriented work could be justified.

The prototype combines recorded-video traffic observation with a simulated urban road network. Video experiments will assess whether detection and tracking can produce dependable counts and queue-related indicators, while the simulator will support fair comparisons among fixed, responsive, and learning-based signal policies. An ambulance mission will use current travel-time and delay estimates to select a valid route. A moving corridor will prepare only the next relevant intersections, complete safe signal transitions, detect passage, and restore normal control. The dashboard will present the network, route, warnings, corridor status, and experiment summaries without granting direct control over public infrastructure. Success will be judged through matched scenarios that report waiting time, queue length, travel time, throughput, fairness, emergency benefit, ordinary-traffic disruption, and recovery. Configurations, software versions, seeds, failures, and measurement definitions will be retained so that comparisons can be checked and repeated. The present deliverable defines the project vision, research basis, requirements, proposed method, and system design. Implementation choices that depend on the final road network, approved video, available hardware, and baseline profiling will be resolved before experimental results are reported.

Table of Contents

List of Figures **ix**

List of Tables **x**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 1

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Conclusion 2

2 Project Vision **3**

2.1 Problem Domain Overview 3

2.2 Problem Statement 4

2.3 Problem Elaboration 5

2.4 Goals and Objectives 5

2.5 Project Scope 6

2.6 Sustainable Development Goals 6

2.7 Constraints and Assumptions 7

2.8 Business and Practical Opportunity 7

2.9 Stakeholders and User Characteristics 8

2.10 Conclusion 9

3 Literature Review / Related Work **10**

3.1 Definitions, Acronyms, and Abbreviations 10

3.2 Detailed Literature Review 11

3.2.1 Vision-Based Traffic Observation 11

3.2.2 Conventional and Adaptive Signal Control 13

3.2.3 Reinforcement Learning and Simulation 14

3.2.4 Dynamic Routing and Emergency Priority 16

3.3 Literature Review Summary Table 17
3.4 Conclusion 22

4 Software Requirement Specifications 23
4.1 List of Features 23
4.2 Functional Requirements 24
4.3 Quality Attributes 25
4.3.1 Usability 26
4.3.2 Reliability and Integrity 26
4.3.3 Security and Confidentiality 27
4.3.4 Performance 27
4.3.5 Maintainability and Reusability 27
4.3.6 Extensibility and Traceability 28
4.4 Non-Functional Requirements 28
4.5 Assumptions 30
4.6 Use Cases 31
4.7 Hardware and Software Requirements 31
4.7.1 Hardware Requirements 36
4.7.2 Software Requirements 37
4.8 Graphical User Interface 37
4.9 Database Design 42
4.9.1 Entity Relationship Diagram 42
4.9.2 Data Dictionary 44
4.10 Risk Analysis 46
4.11 Conclusion 48

5 Proposed Approach and Methodology 49
5.1 Overall Research Workflow 49
5.2 Network and Scenario Preparation 50
5.2.1 Network Provenance and Validation 50
5.2.2 Demand and Incidents 50
5.3 Video Perception Pipeline 51
5.4 Normal Traffic Controllers 52
5.4.1 Fixed-Time Controller 52
5.4.2 Rule-Based Adaptive Controller 52

5.4.3 Proposed RL Formulation . . . 53
5.5 Dynamic Graph Routing . . . 54
5.6 Moving Green Corridor . . . 56
5.7 Integrated Control Loop . . . 58
5.8 Experimental Protocol . . . 59
5.8.1 Comparison Matrix . . . 59
5.8.2 Scenario Assignment and Repetition . . . 59
5.8.3 Metric Definitions . . . 60
5.8.4 Reproducibility Package . . . 60
5.9 Threats to Validity . . . 61
5.10 Conclusion . . . 61

6 High-Level and Low-Level Design 62
6.1 System Overview . . . 62
6.1.1 Operator Interface and Commands . . . 62
6.1.2 Traffic Observation and Simulation . . . 63
6.1.3 Adaptive Control and Emergency Routing . . . 63
6.1.4 Experiment and Evaluator Interaction . . . 64
6.2 Design Considerations . . . 66
6.2.1 Assumptions and Dependencies . . . 66
6.2.2 General Constraints . . . 66
6.2.3 Goals and Guidelines . . . 67
6.2.4 Development Methods . . . 67
6.3 System Architecture . . . 68
6.3.1 High-Level Partitioning . . . 69
6.3.2 Component Collaboration . . . 69
6.3.3 Rationale for Decomposition . . . 70
6.3.4 System Architecture Diagram . . . 70
6.3.5 Subsystem Architecture . . . 70
6.4 Architectural Strategies . . . 71
6.4.1 Choice of Technologies . . . 71
6.4.2 Reuse of Existing Software Components . . . 71
6.4.3 Future Extension and Enhancement . . . 72
6.4.4 Version Control . . . 72
6.4.5 User Interface Paradigm . . . 72

6.4.6 External Database and Data Storage Management 73
6.4.7 Concurrency and Synchronization 73
6.5 Domain Model/Class Diagram 74
6.6 Sequence Diagrams 74
6.6.1 Normal Control Update 74
6.6.2 Ambulance Dispatch and Corridor Activation 75
6.6.3 Rerouting and Recovery 76
6.7 Policies and Tactics 76
6.7.1 Corridor State and Safety Policy 76
6.7.2 Interface and Event Contracts 77
6.7.3 Resource Ownership Tactics 79
6.7.4 Error Handling and Recovery 79
6.7.5 Implementation Policies 80
6.7.6 Testing Strategy 82
6.7.7 Coding, Build, and Maintainability Policies 82
6.7.8 Continuous Integration and Deployment 83
6.7.9 Source Code Organization 83
6.8 Conclusion 84

Bibliography **85**

List of Figures

2.1 Manzil project context and the separation of evidence paths 4

4.1 Proposed operator use-case diagram 32
4.2 Proposed primary navigation flow between Manzil screens 38
4.3 G02 traffic-overview screen wireframe 39
4.4 G03 perception and intersection-detail screen wireframe 40
4.5 G04 emergency-dispatch and route screen wireframe 40
4.6 G01 experiment-configuration screen wireframe 41
4.7 G05 comparative-analytics screen wireframe 41
4.8 Complete grouped conceptual ER model using entity, attribute, and relationship symbols 43
4.9 Experiment scenarios, runs, metrics, and artifacts with conceptual attributes 43
4.10 Observation ownership and control-decision evidence with conceptual attributes . . 44
4.11 Emergency missions, route versions, and priority events with conceptual attributes . 45

5.1 Proposed research and evaluation workflow 49
5.2 Proposed traffic-video perception pipeline 51

6.1 Proposed high-level architecture 64
6.2 Proposed domain model/class diagram 74
6.3 Sequence for a normal signal-control update 75
6.4 Sequence for ambulance dispatch and corridor activation 75
6.5 Sequence for route revision, obsolete-priority cancellation, and recovery 76
6.6 Intersection-level moving-corridor state transitions 77

List of Tables

2.1 **Scope boundaries** . . . 6
2.2 **Stakeholder summary** . . . 8

3.1 **Summary of literature review** . . . 18

4.1 **Functional requirements** . . . 24
4.2 **Non-functional requirements and design targets** . . . 29
4.3 **UC-01: Configure a Scenario** . . . 31
4.4 **UC-02: Run and Control a Simulation** . . . 33
4.5 **UC-03: Process Traffic Video** . . . 33
4.6 **UC-04: Inspect Traffic State** . . . 34
4.7 **UC-05: Dispatch an Ambulance** . . . 34
4.8 **UC-06: Monitor Route and Corridor** . . . 35
4.9 **UC-07: Handle a Blockage or Route Change** . . . 35
4.10 **UC-08: Compare and Export Runs** . . . 36
4.11 **Hardware resources for development and demonstration** . . . 37
4.12 **Software technologies and configuration responsibilities** . . . 38
4.13 **Proposed graphical user interface screen inventory** . . . 39
4.14 **Data dictionary** . . . 45
4.15 **Risk register** . . . 47

5.1 **Planned experiments and primary evidence** . . . 59

6.1 **Component responsibility allocation** . . . 65
6.2 **Architectural decision record** . . . 67
6.3 **Core interface and data-contract fields** . . . 78
6.4 **Failure handling strategy** . . . 80

Chapter 1 Introduction

1.1 Purpose of this Document

This Deliverable 2 report translates the approved proposal into a traceable research and software design. It defines the questions to be investigated, reviews the evidence behind major design choices, specifies externally visible requirements, and describes the methodology and architecture needed to conduct fair experiments. It also records the distinction between confirmed project scope, proposed technical choices, and future implementation work. The report does not claim experimental improvements, real-time performance, model accuracy, or deployment readiness. At the proposal stage, the problem definition, major functions, and conceptual architecture were established, while dataset preparation, implementation, tuning, and evaluation remained planned work. The methodology therefore defines how later claims must be produced rather than anticipating their outcome. This distinction is important because a successful simulation run or a plausible interface design does not demonstrate a measurable improvement. Any later performance statement must be tied to a recorded configuration, matched baseline, stated metric, and repeatable set of runs.

1.2 Intended Audience

The primary audience is the FAST–NUCES FYP supervisor and evaluation panel. They require enough detail to judge the relevance of the evaluation objectives, the feasibility of the proposed software, and the validity of the experimental plan. The report is also an engineering reference for the student implementation team. Team members should be able to derive component responsibilities, interface fields, scenario configurations, fallback behavior, and planned verification from the chapters that follow. A secondary audience consists of prospective traffic-management and emergency-service stakeholders who may wish to understand the problem, operational assumptions, and limits of the prototype. These stakeholders are not represented as current clients, project partners, or approving authorities. The writing therefore explains domain terms when they first become important, but it retains the precision needed for implementation and academic review. Business-level implications are kept separate from technical evidence, and proposed functions are not described as operational capabilities. This distinction helps each audience find relevant information without confusing a research prototype with an operational traffic-control system.

1.3 Definitions, Acronyms, and Abbreviations

Key terms and abbreviations used throughout the report are defined below.

A*: A shortest-path search using accumulated cost and an admissible lower-bound heuristic [1].

API: Application Programming Interface.

CV: Computer vision used to derive observable traffic indicators from video.

Green corridor: A moving sequence of bounded signal-priority interventions ahead of an ambulance.

MOT: Multi-object tracking, which associates detections into identities across video frames.

Queue proxy: An estimate of tracked slow or stationary vehicles in a defined lane region.

RL: Reinforcement learning, in which a policy is optimized using states, actions, and cumulative reward.

Signal preemption: A controlled signal change for an emergency request, including transition and recovery.

SUMO: Simulation of Urban MObility, a microscopic road-traffic simulator.

TraCI: Traffic Control Interface for reading and changing a running SUMO simulation.

Traffic-state contract: A versioned observation structure with source, time, units, confidence, and validity.

Throughput: Vehicles leaving a measured movement or network region per unit time.

Waiting time: Time accumulated below the configured SUMO speed threshold.

1.4 Conclusion

This chapter has established Manzil as a reproducible investigation of interacting traffic decisions rather than a collection of unrelated artificial-intelligence features. The system will study how observable traffic conditions can inform signal control, how dynamic routing can respond to network delay, and how an ambulance can receive temporary priority without ignoring ordinary traffic. It has also fixed the evidence boundary for this deliverable. The approved proposal confirms the problem, intended functions, simulation basis, and broad technology direction, while the repository does not yet provide an implemented controller, trained perception model, prepared road network, or experimental results. Architecture diagrams therefore describe the system to be built, and performance requirements remain targets for later measurement. The defined terminology prevents video-derived proxies from being confused with exact simulator state and distinguishes a moving corridor from an unsafe instruction to turn every signal green. A proposed choice may be described as implemented only after its code, configuration, test evidence, or experimental record exists.

Chapter 2 Project Vision

This chapter converts the broad motivation for Manzil into a bounded project vision that can guide both implementation and evaluation. It identifies the decisions that the prototype is expected to make, the evidence needed to judge those decisions, and the conditions under which any conclusion must remain limited to simulation and selected recorded video. The chapter begins with the relationship between normal traffic control and emergency movement, then states the problem in terms that can be investigated within an undergraduate project. The evaluation plan separates perception quality, normal signal control, dynamic routing, corridor operation, and robustness so that an improvement in one area is not mistaken for success in every area. The objectives and scope retain the approved proposal while excluding forecasting, enforcement, citywide deployment, and direct operation of public signals. Constraints related to data, map quality, computing resources, and the simulation-to-reality gap are treated as design inputs rather than footnotes. Stakeholder expectations then clarify who can use the prototype and what evidence would constitute a credible project outcome.

2.1 Problem Domain Overview

Traffic-signal control allocates limited green time among movements that cannot proceed together. Fixed plans are understandable, auditable, and easy to reproduce, but they cannot respond to every short-term difference in approach demand. Adaptive control can use measured queues and waiting time, yet a controller that improves the network average may still be unacceptable if it repeatedly postpones a lightly loaded approach. Coordination adds another layer because an action at one intersection changes arrivals and queues downstream. Emergency routing operates on the same network with a stricter travel-time objective. A route is useful only when its cost reflects current link travel time, queue discharge, signal delay, and any closure that makes a movement unavailable. Signal priority may reduce ambulance stops, but an unsafe transition or poorly managed recovery can transfer excessive delay to ordinary traffic. Manzil therefore treats normal control, route selection, priority activation, passage detection, and recovery as connected decisions. It retains separate measures for the ambulance and other vehicles so that emergency benefit cannot conceal network disruption.

Figure 2.1 presents the central scope decision that separates the two evidence paths while allowing them to use the same traffic-state contract. Recorded traffic video enters the perception path, where a detector, tracker, and region-assignment process produce observable measures that can be checked against annotations. SUMO enters the controlled-experiment path and exposes exact simulated vehicle, lane, route, and signal state for repeatable controller and routing comparisons. The shared contract records the source mode, identifier, timestamp, unit, confidence, and validity of every observation. This design per-

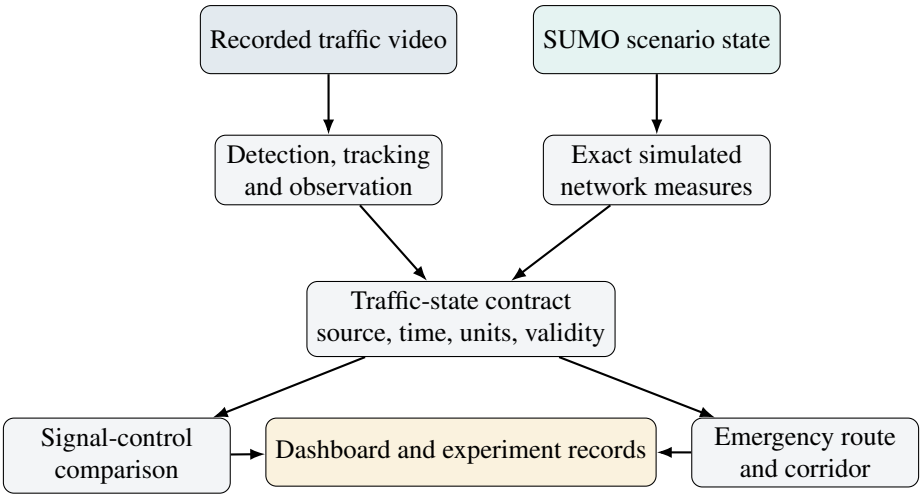


Figure 2.1: Manzil project context and the separation of evidence paths

mits the dashboard and downstream services to consume a consistent structure without implying that the sources have equal certainty. In particular, a count or queue proxy estimated from an arbitrary camera view cannot reconstruct every lane in the selected SUMO network unless camera calibration and lane mapping have been completed. The figure is relevant because it prevents a common scope error: presenting separate perception and simulation demonstrations as though they formed a validated real-time city model. Manzil will evaluate them separately unless a later, explicitly controlled integration study becomes feasible.

2.2 Problem Statement

Within the selected prototype setting, there is no established implementation that allows the Manzil team to compare signal-control strategies and isolate the contributions of emergency routing and coordinated priority under one reproducible protocol. The project problem is therefore to design a system that observes or simulates traffic state, makes bounded control and routing decisions, and records why each decision was requested and what was actually applied. The system must support matched comparisons between fixed-time, rule-based adaptive, and reinforcement-learning control under the same demand conditions. It must also compare an unassisted ambulance, congestion-aware routing, and routing with a moving corridor without changing unrelated configuration between runs. This requires more than joining familiar technologies. Measurements must have defined units and freshness, routes must remain legal from the current vehicle position, signal changes must respect phase safety, and failed or infeasible runs must remain visible. The resulting evidence will describe performance in declared simulation and video conditions. It will not establish that the same effect would occur on public roads without calibration, controller integration, safety engineering, authority approval, and field validation.

2.3 Problem Elaboration

Five linked sub-problems determine whether the prototype can meet its evaluation objectives. First, frame-level vehicle detections must be associated over time and assigned to meaningful road regions before they can support counts, trajectories, or queue-related observations. Second, normal signal control requires a correctly configured fixed baseline and a transparent adaptive baseline before reinforcement learning can be judged fairly. Third, route costs must use compatible units and avoid adding separate congestion penalties when observed travel time already contains the same delay. Fourth, corridor control must anticipate the ambulance, clear conflicting movements, limit the duration of priority, detect passage, and restore normal service without holding the entire route green. Fifth, experiments must use matched demand, seeds, warm-up, duration, and metric definitions. These issues also interact across component boundaries. A stale observation can change an edge cost, a route revision can invalidate pending priority, and a prolonged priority phase can improve ambulance time while harming ordinary traffic. The vision consequently requires explicit freshness, single ownership of simulation time and actuation, safe fallback, route versioning, and auditable event order.

2.4 Goals and Objectives

The goal is to develop and evaluate a simulation-based intelligent traffic-management prototype that supports normal traffic control and a single emergency mission. The measurable objectives are:

O1. Define and implement a versioned traffic-state representation shared by perception, simulation, control, routing, and visualization.

O2. Evaluate a pretrained vehicle detector with multi-object tracking on held-out, source-separated traffic-video segments.

O3. Construct a manageable multi-intersection SUMO network and repeatable demand scenarios.

O4. Implement fixed-time and reproducible rule-adaptive controllers, followed by one feasible RL controller.

O5. Compute valid ambulance routes using time-based dynamic edge costs and bounded rerouting.

O6. Coordinate safe, moving signal priority and restore normal control after passage or cancellation.

O7. Provide a React dashboard and FastAPI service boundary for scenario control, monitoring, and experiment comparison.

O8. Record configurations and outcomes so each reported run can be reproduced and audited.

2.5 Project Scope

Table 2.1: Scope boundaries

In scope	Outside the current scope
Recorded-video vehicle detection, tracking, counts, and queue proxies	Live government CCTV integration or citywide camera calibration
Selected multi-intersection SUMO network and synthetic demand	All of Lahore or safety-certified field deployment
Fixed, rule-adaptive, and one RL controller	Replacing real traffic-controller firmware
Single active ambulance with dynamic routing and moving priority	Multiple competing emergency fleets as a core case
Simulated closure or incident as a routing trigger	Visual accident classification or traffic forecasting
React dashboard, FastAPI services, and experiment records	Native mobile application, enforcement, facial or number-plate recognition
OpenStreetMap import with manual cleanup where feasible	A guarantee that map attributes are complete or current

Table 2.1 separates the functions required for the research prototype from capabilities that would create unsupported operational or regulatory claims. The in-scope work is sufficient to examine the approved project idea: recorded-video traffic perception, a manageable multi-intersection simulation, three levels of normal signal control, congestion-aware routing for one ambulance, a moving priority corridor, and a dashboard with experimental records. The exclusions are equally important. Live government CCTV, direct operation of public signals, full-Lahore coverage, enforcement, number-plate or facial recognition, visual accident classification, forecasting, and multiple competing emergency fleets would require data, infrastructure, policy, and validation beyond the current project. A road closure may still be introduced as a controlled simulation event because it is needed to test rerouting, but the system will not claim to detect that incident visually. OpenStreetMap supports network preparation, yet its attributes will be checked and corrected rather than assumed complete. These boundaries keep implementation feasible and ensure that the final evaluation addresses the stated objectives instead of dispersing effort across unrelated features.

2.6 Sustainable Development Goals

Manzil aligns principally with United Nations Sustainable Development Goal 11, which concerns inclusive, safe, resilient, and sustainable cities [2]. The relevant connection is the investigation of how existing road and signal capacity might be used more responsively under changing traffic conditions, together with a structured method for prioritizing an emergency journey. The prototype can provide evidence about simulated waiting time, queue formation, throughput, ambulance travel time, and the re-

covery of ordinary traffic after priority. These measures are appropriate for a computing project because they can be defined and reproduced within the selected environment. The alignment must nevertheless be stated cautiously. A SUMO experiment does not demonstrate city-level sustainability, reduce emissions directly, or establish a public-health outcome. The project also does not model patient condition, clinical triage, hospital capacity, or the relationship between travel time and survival. Its secondary relevance to health is limited to studying a transport mechanism that may reduce simulated ambulance delay. Any stronger claim would require operational traffic data, emergency-service participation, field trials, and outcome measures outside the approved scope.

2.7 Constraints and Assumptions

The main constraints are the semester-scale implementation period, limited access to calibrated local traffic data, camera perspective and occlusion, finite model-training hardware, and the gap between simulated and real driving behavior. These constraints require a network small enough for repeated runs and detailed validation. A recognizable urban extract is preferred, but a documented synthetic grid remains an acceptable fallback if imported lane connections or signal programs cannot be corrected reliably within the schedule. The core experiments assume one active ambulance, valid origin and destination edges, controlled traffic generation, and no direct actuation of public infrastructure. Video-derived queue length remains an observable proxy unless the camera is calibrated to physical road geometry. The exact tracker, reinforcement-learning algorithm, database engine, hardware, map size, timing thresholds, and training budget are proposed choices rather than confirmed facts. These assumptions will be stored with experiment configurations because they affect interpretation. When an assumption fails, the system must report the failure or use a declared fallback instead of silently changing the scenario. This treatment makes constraints part of the research design and prevents later results from appearing more general than the tested conditions support.

2.8 Business and Practical Opportunity

The practical opportunity for Manzil is an experimentation and decision-support environment rather than an immediately deployable traffic product. A repeatable simulator can help a technical team compare how signal policies react to balanced, uneven, congested, and incident-affected demand. It can also show a prospective traffic or emergency-service stakeholder why the fastest ambulance route may differ from the shortest route, how a moving corridor progresses, and how much ordinary traffic is disrupted during and after priority. The dashboard and stored run manifests make those decisions visible instead of presenting the controller as a black box. This capability could support later feasibility studies, training

demonstrations, or policy prototyping if the research results justify continued work. A commercial or public-sector deployment would require substantially more evidence: calibrated local demand, validated sensing, compatible roadside controllers, resilient communications, cybersecurity controls, privacy review, safety assurance, authority approval, and field trials. The current project assumes none of these partnerships or approvals and makes no revenue claim. Its immediate value lies in producing a defensible prototype and evaluation method that can reveal both benefits and limitations.

2.9 Stakeholders and User Characteristics

Table 2.2: Stakeholder summary

Stakeholder	Interest	Expected interaction
Student research team	Correct implementation and defensible experiments	Configure, run, diagnose, export, and compare experiments
Supervisor and evaluators	Scope, rigor, traceability, and evidence quality	Review report, configurations, demonstrations, and results
Prototype operator	Clear scenario and emergency controls	Use the dashboard; inspect warnings and system state
Prospective traffic authority	Safety, reliability, network impact, auditability	Future consultation only; no current operational role
Prospective emergency service	Travel-time benefit and predictable priority behavior	Future requirements input only
Road users	Fairness and limited disruption	Indirect beneficiaries or affected parties in simulated evaluation

Table 2.2 identifies the parties who influence the prototype or may later be affected by its decisions. The student team is the active system operator during development and evaluation, with responsibility for scenario configuration, execution, diagnosis, and evidence preservation. The supervisor and evaluation panel require transparent scope, reproducible methods, and a clear distinction between implemented and planned work. A prototype operator needs understandable controls and visible warnings but is not treated as an authorized emergency dispatcher. Traffic authorities and emergency services are prospective stakeholders whose operational knowledge would be necessary for any future deployment; they are not represented as current partners or as having approved the design. Road users are indirect affected parties because priority can shift delay between approaches even when an ambulance benefits. This stakeholder distinction shapes the interface and evaluation. The dashboard must expose source mode, stale data, fallback, route revisions, and corridor state to the operator, while experimental reporting must include ordinary-traffic effects for reviewers and prospective authorities. No stakeholder is assigned a role, credential, or commitment that is unsupported by the project record.

2.10 Conclusion

This chapter has defined a project vision that is narrower than operational traffic management but broad enough to support a substantive research and development investigation. Manzil will connect traffic observation, comparative signal control, dynamic emergency routing, and a moving green corridor inside a controlled and auditable prototype. The two evidence paths remain separate, and the core emergency case contains one active ambulance so that route and priority behavior can be examined without inventing an untested fleet-arbitration policy. The objectives identify the controlled comparison and corresponding measures for each claim. Scope boundaries exclude live government integration, citywide deployment, forecasting, enforcement, and unsupported safety or health conclusions. Constraints concerning data, map quality, hardware, and simulation fidelity are incorporated into configuration and reporting. Stakeholder roles are limited to those supported by the project record, and credible evaluation requires legal behavior, reproducible runs, visible failure, and balanced reporting of benefits and side effects. These decisions guide the literature review in Chapter 3, which examines the research basis for each technical component and identifies the limitations that Manzil's methodology must address.

Chapter 3 Literature Review / Related Work

This chapter examines the research needed to justify Manzil's design rather than treating the use of familiar tools as a contribution. The review is organized around five decisions: how traffic is observed, how normal signals are controlled, how learned policies are evaluated, how emergency routes are selected, and how signal priority is activated and recovered. Each study is considered in its original data or simulation setting because reported values from incompatible benchmarks cannot be treated as a common comparison. The discussion identifies the method, evidence, limitations, and direct implication for a Manzil design choice. Research papers are distinguished from official software documentation, which is used to establish interface capabilities but is not counted toward the study requirement. The purpose is not to assemble every publication on intelligent transportation. It is to establish whether the proposed components are technically defensible, which assumptions must be recorded, and where a controlled undergraduate experiment can add useful evidence. The final synthesis consequently states a precise integration question instead of claiming that the use of YOLO, reinforcement learning, or shortest-path search is itself novel.

3.1 Definitions, Acronyms, and Abbreviations

Additional terms used specifically in the literature review are defined below.

DQN: Deep Q-Network, a value-based reinforcement-learning method that approximates action values with a neural network.

IDF1: Identity F1 score, a tracking measure based on correctly identified detections across a sequence.

IoU: Intersection over Union, the overlap ratio commonly used when matching a predicted bounding box with a reference box.

mAP: Mean Average Precision, a detection measure aggregated across classes or overlap thresholds under a stated protocol.

MARL: Multi-agent reinforcement learning, in which several decision-making agents learn within a shared environment.

MDP: Markov Decision Process, a formal model of states, actions, transitions, and rewards used to define an RL problem.

OSM: OpenStreetMap, the collaborative geographic data source considered for preparing the SUMO road network.

YOLO: You Only Look Once, a family of one-stage object-detection methods.

3.2 Detailed Literature Review

The approved proposal supplied the initial bibliography, after which each retained study was checked against a publisher page, proceedings record, DOI, or official open-access paper. Selection required direct relevance to vehicle detection and tracking, traffic-state estimation, signal control, simulation, graph routing, or emergency preemption. The review also applies consistent terminology. Detection refers to locating and classifying vehicles within one frame, while tracking refers to associating those observations through time. A detector therefore cannot produce reliable passage counts by itself. Signal priority describes preferential treatment that may remain within normal timing constraints, whereas preemption may interrupt normal operation for an emergency request. Manzil uses the term moving green corridor for a coordinated sequence of bounded interventions ahead of the ambulance, not for a permanently green route. These distinctions affect the interpretation of prior results and the design of the proposed experiment. A paper reporting benchmark detection accuracy does not establish a calibrated queue estimate, and a study of isolated preemption does not automatically validate network-level routing. The following sections preserve those boundaries when relating the literature to Manzil.

3.2.1 Vision-Based Traffic Observation

3.2.1.1 Research Overview

Redmon et al. formulated YOLO as a single network that predicts bounding boxes and class probabilities from an image in one evaluation [3]. The study established the feasibility of real-time one-stage detection and reported a strong speed advantage, while also identifying localization error as a relative weakness. The design is relevant to Manzil because a compact inference path is practical for recorded traffic video and may later support a live demonstration on available hardware. The evidence does not establish accuracy for every vehicle class, road layout, camera height, weather condition, or Lahore traffic scene. A selected pretrained checkpoint must therefore be evaluated on the project's own held-out sequences, and confidence or intersection-over-union thresholds cannot be copied from an unrelated benchmark. Frame-level boxes also do not provide stable counts by themselves. The same vehicle may be detected in many frames, an occluded vehicle may disappear temporarily, and perspective changes apparent size across the image. Manzil consequently treats one-stage detection as the first part of a measurement pipeline that also requires tracking, road-region assignment, temporal aggregation, and explicit uncertainty.

3.2.1.2 Critical Assessment

Wojke et al. extended a simple online tracker with a learned appearance metric to improve association through longer occlusions [4]. Appearance information can reduce identity switches, but it adds a feature-extraction model, processing cost, and sensitivity to domain differences. ByteTrack follows a different strategy by associating high-confidence detections first and then attempting to recover true objects from lower-confidence boxes rather than discarding them [5]. The authors evaluated it across several tracking benchmarks and reported strong accuracy with real-time throughput on their stated hardware. This approach is relevant to traffic video because occlusion and motion blur often reduce detection confidence. Neither benchmark establishes Manzil's processing speed, lane assignment, or queue accuracy. An identity-preserving track still requires a road-contact point, direction rule, and region geometry before it becomes a valid traffic observation. These studies therefore motivate an empirical tracker choice based on held-out count error, identity stability where identities are needed, failure under occlusion, and end-to-end throughput on the team's hardware. If a simpler association method produces equally reliable aggregate counts, its lower complexity may be preferable.

3.2.1.3 Relevance to Manzil

BDD100K provides a large and diverse driving dataset with detection, tracking, and other driving-scene annotations collected across varied locations, weather, and times [6]. Its diversity is useful for model initialization and stress testing, although its primarily vehicle-mounted viewpoint differs from a fixed roadside or elevated camera. UA-DETRAC focuses directly on vehicle detection and tracking in traffic-surveillance video, making its camera geometry and evaluation protocol closer to an intersection-observation task [7]. Its scenes still cannot be assumed to represent Lahore traffic composition, motorcycle and rickshaw prevalence, lane discipline, lighting, or camera placement. The two datasets are therefore complementary references rather than automatic substitutes for local validation. Manzil should use licensed public sequences where they support reproducible evaluation and may add a small, separately documented local sample if collection permission and annotation effort are feasible. These datasets also influence the split strategy. Adjacent video frames are highly correlated, so a random frame split can place nearly identical observations in training and test data. The project will split by source or continuous sequence, retain a held-out evaluation set, and avoid tuning thresholds on that set.

3.2.2 Conventional and Adaptive Signal Control

3.2.2.1 Research Overview

A fixed-time controller remains necessary even when it is not expected to dominate every scenario. It provides a stable reference against which the value of traffic-responsive decisions can be measured, and it helps reveal whether demand generation or metric collection is biased before more complex controllers are introduced. The baseline must be credible rather than deliberately weak. Manzil will therefore document its phase sequence, green times, yellow and all-red clearance, cycle length, offsets where relevant, minimum service, and the demand conditions used to choose those settings. The same phase definitions and safety transitions will be shared with adaptive controllers so that a comparison does not reward one controller for using an illegal or more permissive movement set. Fixed timing is also useful for route and corridor ablations because its behavior is deterministic and easy to reproduce. If the fixed plan is severely under-saturated or over-saturated in every scenario, differences in controller performance will be difficult to interpret. A basic capacity and queue check will therefore confirm that the scenario family contains meaningful light, medium, and heavy conditions instead of a single demand level chosen to favor adaptation.

3.2.2.2 Critical Assessment

Varaiya formalized max-pressure control for a network of signalized intersections using differences between upstream and downstream queue conditions [8]. Selecting a phase that reduces network pressure has a clearer transportation basis than combining arbitrary percentages without explaining what the score represents. The approach is responsive and can operate from local movement information, which makes it a valuable non-learning reference for Manzil. Its assumptions still matter. Queue observations must correspond to valid movements, downstream storage must be represented, and a theoretical selection rule does not remove practical minimum-green, yellow, all-red, or maximum-red constraints. Manzil's initial rule-based adaptive controller will therefore use a pressure-inspired score only when the required upstream and downstream measures are available and sufficiently fresh. Waiting time or maximum-red information will remain as an explicit fairness safeguard so that a low-volume approach is not postponed indefinitely. The controller will also provide a reason for each request, allowing reviewers to see which queues and constraints influenced the decision. This transparency makes the baseline useful even if the later reinforcement-learning policy does not outperform it.

3.2.2.3 Relevance to Manzil

The conventional-control literature determines both the baseline design and the limits of any later comparison. Manzil will implement a documented fixed plan first because it provides a stable reference for validating demand, signal transitions, and metric collection. A pressure-inspired controller will then respond to current movement conditions while using the same legal phases, yellow intervals, all-red clearance, and minimum-service rules. This shared safety envelope prevents the adaptive method from gaining an artificial advantage through a more permissive phase program. The rule-based controller must also expose the queues, downstream conditions, waiting time, and constraint that produced each requested action. Such records make an unexpected result diagnosable and provide a meaningful comparison for a learned policy. Evaluation will cover light, medium, and heavy demand with matched seeds rather than one scenario selected after observing the outcome. Network averages will be accompanied by maximum queues and maximum-red or fairness measures, since a lower mean delay can hide repeated postponement of a lightly loaded approach. These decisions convert the reviewed theory into an auditable baseline rather than an informal demonstration.

3.2.3 Reinforcement Learning and Simulation

3.2.3.1 Research Overview

Liang et al. used deep reinforcement learning to adjust traffic-light cycle behavior from observed traffic state [9]. The study demonstrates how an action definition, decision interval, state representation, and reward can be joined into a trainable signal controller. It also illustrates a limitation common to traffic RL: performance obtained for a particular intersection, simulator configuration, demand pattern, and training budget cannot be assumed to transfer to a different network. Manzil will therefore not reproduce a reported improvement as an expected project target or select deep Q-learning merely because it is familiar. A value-based method is feasible only after the discrete action space is bounded, legal phase transitions are enforced outside the learned policy, and fixed and rule-based baselines behave correctly. State normalization and reward components must be declared before evaluation, while training and tuning scenarios must remain separate from held-out tests. The policy will be assessed using traffic measures and safety interventions, not training return alone. If a simpler controller performs as well under the selected scenarios, that result will be reported rather than interpreted as a failure to produce a sufficiently complex model.

3.2.3.2 Critical Assessment

PressLight connects reinforcement-learning state and reward design to the transportation concept of max pressure [10]. The paper argues that a theoretically grounded pressure signal can reduce reliance on arbitrary handcrafted objectives and evaluates the method against conventional and learning baselines in arterial networks. This is relevant to Manzil because a reward based only on total waiting time may conceal spillback, unbalanced approaches, or repeated postponement of a minor movement. Pressure can provide a principal network-efficiency signal, while explicit penalties or constraints can address starvation and unnecessary switching. The study does not remove the need for careful evaluation. Topology, movement mapping, flows, phase definitions, observation range, and training budget all influence the learned result. Manzil must therefore record the exact state features, normalization, action timing, reward components, and safety interventions used by the selected policy. Emergency priority will initially remain a supervisory state machine rather than a rare reward event folded into normal training. That separation permits the normal-controller comparison to be interpreted independently and lets the additional effect of the corridor be measured through a controlled ablation.

FRAP represents competition among signal phases and exploits structural relationships so that equivalent movements need not be learned as unrelated cases [11]. This is useful where an intersection contains several competing phases, but it requires an accurate phase graph and a more specialized representation. CoLight addresses cooperation across intersections through graph attention, allowing neighboring conditions to influence an agent's decision [12]. Chu et al. similarly study decentralized multi-agent deep reinforcement learning at larger scale and emphasize the importance of communication and partial observability [13]. Together, these studies show why the phrase "multi-intersection RL" is not a complete method. A project must state whether one centralized policy chooses all actions, independent agents operate without communication, or cooperating agents exchange defined observations. Multi-agent learning also introduces non-stationarity because each agent experiences an environment affected by policies that are changing at the same time. For Manzil, attention-based coordination may exceed the available tuning budget and make attribution difficult. The initial design will therefore use a small network and one explicit ownership model, adopting useful phase-structure ideas without claiming to reproduce every cited architecture.

3.2.3.3 Relevance to Manzil

CityFlow was introduced as a reinforcement-learning environment designed for large-scale city traffic and high simulation throughput [14]. It is an important alternative when training speed and very large networks dominate the design. SUMO is retained for Manzil because the approved proposal already de-

pends on its microscopic vehicle model, traffic-light programs, OpenStreetMap conversion tools, NetEdit workflow, and TraCI interface. Lopez et al. describe the simulator's architecture and its support for detailed, externally controlled experiments [15]. This combination is appropriate for a manageable multi-intersection prototype in which route validity, individual ambulance movement, signal transitions, and per-vehicle outcomes must be inspected. Retaining SUMO also avoids the additional work of showing that a migrated network and demand model behave equivalently in another simulator. The choice does not make simulation results equivalent to field evidence. Car-following parameters, lane connections, signal programs, vehicle insertion, and map cleanup can materially change outcomes. Manzil will therefore version the network, demand, routes, simulator settings, and software release, and it will limit conclusions to the declared simulation conditions.

3.2.4 Dynamic Routing and Emergency Priority

3.2.4.1 Research Overview

Dijkstra's algorithm finds a minimum-cost path on a graph whose edge weights are nonnegative [16]. Hart, Nilsson, and Raphael established the formal basis of A*, which adds a heuristic lower-bound estimate to guide the search [1]. Both methods are well established, so selecting either algorithm is not a research contribution by itself. Manzil's project-specific task is to build a graph that respects valid SUMO vehicle movements and to define a cost that reflects time rather than distance alone. The proposed cost combines link travel time with non-overlapping estimates of queue discharge and residual signal delay, all expressed in seconds. If measured link travel time already includes queue delay, a separate congestion term cannot be added without double-counting. Dijkstra will provide the reference route, while A* is permitted only when free-flow travel time forms an admissible lower bound under nonnegative added delay. Dynamic rerouting also needs stability. A route will change when the current path becomes infeasible or when an alternative offers a persistent improvement above a defined threshold. Route revisions, no-route cases, and computation time will be evaluated rather than treated as incidental log messages.

3.2.4.2 Critical Assessment

Humagain et al. systematically reviewed emergency-vehicle route optimization and preemption methods and identified recurring concerns involving dynamic traffic data, travel-time objectives, ordinary-traffic effects, safety, and limited real-world application [17]. The review supports an integrated evaluation but does not justify the unsupported claim that integrated solutions do not exist. Qin and Khan examine transitions into emergency preemption and recovery to normal timing, including clearance and the queues created by interruption [18]. Their work shows that preemption is a lifecycle rather than a single

instruction to set an approach green. Louati et al. investigate queue-aware emergency crossing across signalized intersections [19]. This is relevant because a green signal does not guarantee an unobstructed approach when the standing queue cannot discharge before the emergency vehicle arrives. Manzil therefore models requested, safe-transition, active-priority, passage, recovery, and cancellation states. It also uses look-ahead activation, timeout, downstream checks, and bounded priority duration. The precise timing values remain configurable simulation policies, not certified real-world safety parameters, and the evaluation will report ordinary-traffic disruption together with ambulance benefit.

3.2.4.3 Relevance to Manzil

The literature establishes reliable building blocks while exposing the risks that arise when they are connected. One-stage detection is fast enough to investigate traffic video, but useful traffic indicators require tracking, road geometry, temporal rules, and validation on the intended viewpoint. Max-pressure and reinforcement-learning research provide responsive control strategies, yet their comparison depends on state representation, phase constraints, reward design, training data, and simulator configuration. Dijkstra and A* solve well-defined path problems, but emergency routing remains only as credible as its movement graph, time-based costs, observation freshness, and rerouting stability. Preemption research makes safe transition and recovery central, and the review of emergency methods shows why ordinary-traffic disruption cannot be omitted. Manzil's proposed contribution is consequently not a new detector, shortest-path algorithm, or reinforcement-learning family. It is the controlled evaluation of an integrated and traceable prototype in which perception and simulation evidence remain distinct, normal controllers receive matched scenarios, routing and corridor effects are separated, and ambulance benefit is reported with network disruption and recovery. The unresolved question is whether that integration yields consistent benefits under the selected scenarios and constraints.

3.3 Literature Review Summary Table

Table 3.1 consolidates the evidence by technical theme so that each study retains its evaluation setting and reported contribution. The reported-evidence column records what the authors evaluated rather than converting unlike metrics into a common ranking. The final column states both the boundary of that evidence and the specific decision it supports in Manzil. This organization is important because a detector benchmark, a traffic simulator, and a preemption experiment answer different questions even when each reports an improvement. The summary supports traceability rather than an artificial numerical comparison. It also exposes where Manzil still needs its own evidence: selected video, the final SUMO network, held-out demand, chosen hardware, and recovery behavior. Detailed qualifications remain in the preceding discussion, while the table provides a compact link from verified literature to requirements,

methods, and design choices. The table uses wrapped columns and continues across portrait pages, which preserves legibility without rotating a page or treating unlike evidence as a common numerical ranking.

Table 3.1: Summary of literature review

Study and year	Method and setting	Reported evidence	Boundary and Manzil implication
Redmon et al., 2016 [3]	Unified one-stage object detection evaluated on established image benchmarks.	Demonstrated real-time inference and a useful speed–accuracy tradeoff for its reported models.	Benchmark performance does not establish roadside traffic accuracy; evaluate the selected checkpoint on held-out traffic sequences.
Wojke et al., 2017 [4]	Online tracking augmented with a learned appearance metric, evaluated in pedestrian tracking.	Reduced association failures through longer occlusions in the reported setting.	Appearance features add cost and domain dependence; use only if they improve traffic-count stability.
Zhang et al., 2022 [5]	Two-stage association retains lower-score detections across several tracking benchmarks.	Reported competitive tracking accuracy and real-time throughput on the stated hardware.	Benchmark identities do not imply calibrated traffic measures; compare end-to-end count and queue-proxy error.

Study and year	Method and setting	Reported evidence	Boundary and Manzil implication
Yu et al., 2020 [6]	Large multi-task driving dataset spanning varied locations, weather, and time.	Supplies detection, tracking, and related driving-scene annotations at substantial scale.	Vehicle-mounted views differ from fixed intersection cameras; use for initialization or stress testing, not sole validation.
Wen et al., 2020 [7]	Vehicle detection and tracking benchmark using traffic-surveillance video.	Provides a closer roadside setting and a joint protocol for detector and tracker evaluation.	Site geometry and vehicle mix remain domain-specific; split by sequence and disclose the local-domain gap.
Varaiya, 2013 [8]	Max-pressure control for signalized traffic networks under stated queueing assumptions.	Establishes a principled relationship between movement pressure and network control.	Requires valid upstream and downstream queues; use as a transparent adaptive reference with safety constraints.
Liang et al., 2019 [9]	Deep Q-learning applied to traffic-light cycle control in simulation.	Demonstrates that bounded signal actions can be learned from traffic state in the tested configuration.	Topology and demand transfer are uncertain; train separately and evaluate on held-out scenarios.

Study and year	Method and setting	Reported evidence	Boundary and Manzil implication
Wei et al., 2019 [10]	Reinforcement learning with pressure-informed state and reward on arterial networks.	Reported improvements over selected baselines in the authors' simulated networks.	Results depend on flows and training design; adopt the pressure rationale, not the reported improvement as a target.
Zheng et al., 2019 [11]	Phase-competition representation for traffic-signal reinforcement learning.	Exploits structural relationships among competing phases in its evaluations.	Needs specialized phase encoding; preserve legal phase structure before considering this added complexity.
Wei et al., 2019 [12]	Graph-attention cooperation among intersections in simulated road networks.	Shows how neighbouring signal information can support coordinated decisions in tested settings.	Training and attribution are more complex; keep the first Manzil network small and declare all shared observations.
Chu et al., 2020 [13]	Decentralized multi-agent deep reinforcement learning for larger traffic networks.	Evaluates scalable coordination under partial local observations.	Non-stationarity and tuning cost threaten reproducibility; begin with a parameter-sharing policy and stable baselines.

Study and year	Method and setting	Reported evidence	Boundary and Manzil implication
Zhang et al., 2019 [14]	High-throughput simulator designed for city-scale traffic-learning experiments.	Demonstrates an alternative platform for large parallel experiments.	Migration would conflict with the approved SUMO-centered scope; retain only as a documented alternative.
Lopez et al., 2018 [15]	Microscopic traffic simulator with external-control support and network tooling.	Documents the simulator architecture used for detailed traffic and control experiments.	Results inherit network and car-following assumptions; version every map, demand file, and simulator setting.
Dijkstra, 1959 [16]	Minimum-cost search on graphs with nonnegative edge weights.	Provides a reference optimality guarantee for the defined cost snapshot.	It is not dynamic unless costs are refreshed; use as the correctness baseline for versioned routes.
Hart et al., 1968 [1]	Heuristic minimum-cost search using an admissible lower bound.	Reduces unnecessary exploration while retaining optimality under the stated conditions.	An inadmissible heuristic loses the guarantee; use free-flow time as a lower bound or retain Dijkstra.
Humagain et al., 2020 [17]	Systematic review of emergency routing and traffic-signal preemption research.	Identifies recurring objectives, data requirements, safety concerns, and limits in real-world validation.	A review validates no single implementation; measure ambulance benefit and ordinary-traffic disruption together.

Study and year	Method and setting	Reported evidence	Boundary and Manzil implication
Qin and Khan, 2012 [18]	Simulation study of transition into preemption and recovery to normal timing.	Shows that entry, queue effects, and recovery policies affect the outcome of emergency intervention.	Reported timings are not certified for Manzil; implement an explicit guarded lifecycle and evaluate recovery.
Louati et al., 2018 [19]	Queue-aware multi-agent method for emergency crossing at signalized intersections.	Evaluates priority decisions that account for standing traffic rather than green state alone.	Its multi-vehicle scope exceeds the core project; retain queue awareness and downstream-capacity checks.

3.4 Conclusion

The review supports a staged technical strategy. Fixed-time and pressure-inspired controllers must operate correctly before comparison with a learned controller, and measurement definitions must be fixed before an optimization result is interpreted. Video experiments should assess the complete observation chain rather than quote detector accuracy as a traffic-management result. Simulation experiments should use matched seeds, identical safety constraints, and held-out scenarios so controller differences are not confused with different demand. Emergency evaluation must separate route selection from signal priority because combining both changes would obscure the source of any improvement. The cited literature justifies the selected building blocks but does not establish their performance for Manzil's network, cameras, or computing resources. It also requires results to remain bounded to the tested viewpoints, traffic demand, simulator settings, and route conditions. Chapter 4 converts these findings into traceable requirements, while Chapter 5 defines the measurements and controlled comparisons needed to address the evaluation objectives without claiming field deployment or certified safety evidence.

Chapter 4 Software Requirement Specifications

This chapter translates the reviewed problem and evidence into a specification for the Manzil research prototype. The requirements describe observable behavior rather than implying that every item has already been implemented. Where the available repository evidence does not support a measured threshold, the text states a design target or requires profiling before a value is fixed. The specification gives particular attention to reproducibility, legal signal transitions, observation freshness, route-version consistency, and visible failure handling because these properties determine whether later experiments can be interpreted. It also separates the video-perception workflow from SUMO-based control experiments so that estimated observations are not silently presented as simulator ground truth. Account management, public dispatch integration, traffic enforcement, clinical response prediction, and physical signal deployment remain outside the approved scope. The resulting requirements provide a contract for the architecture in Chapter 6 and a traceable basis for the evaluation procedures in Chapter 5.

4.1 List of Features

The prototype is organized into six feature groups that follow the operator's experimental workflow from configuration to evidence export. The grouping prevents a user-interface screen or implementation library from being mistaken for a requirement and clarifies which service owns each outcome. It also shows that emergency operation is not an isolated routing feature: a valid mission depends on scenario state, traffic observations, signal control, monitoring, and durable records. The following statements summarize those groups before the individual functional requirements define testable behavior.

1. The operator can select a network, demand profile, controller, seed, incident, duration, and optional emergency mission before a run begins.
2. The perception workflow can process licensed recorded video into detections, tracks, road-region observations, and timestamped aggregate measures with confidence information.
3. The simulation workflow advances SUMO through one owner, reads traffic state, applies fixed or adaptive requests, and preserves legal phase transitions.
4. The emergency workflow validates endpoints, computes a dynamic path, applies stable rerouting, activates bounded moving priority, detects passage, and recovers normal control.
5. The monitoring interface presents map state, signals, traffic indicators, ambulance route, corridor state, warnings, and the current data-source mode.
6. The experiment service retains immutable configuration, events, metrics, software versions, seeds,

and termination state for comparison and export.

4.2 Functional Requirements

Table 4.1 defines the mandatory system behavior using stable identifiers that are reused in the traceability matrix. The requirements distinguish a requested signal action from the action actually applied, require route and observation versions to be retained, and make fallback behavior part of the specification rather than an informal implementation detail. Several entries use the word "shall" because they are acceptance conditions for the proposed prototype; they are not claims of completed implementation. Together, FR-01 through FR-08 cover experiment configuration and normal control, FR-09 through FR-14 define the emergency-route and corridor lifecycle, and FR-15 through FR-20 cover monitoring, evidence, comparison, export, and controller failure. This arrangement lets tests target a coherent workflow while preserving individual identifiers for defect tracking. A requirement is satisfied only when its normal and failure paths are demonstrated with retained evidence. For example, returning a path is insufficient for FR-10 if the cost snapshot and graph-validity conditions are not also recorded.

Table 4.1: Functional requirements

ID	Requirement
FR-01	The system shall validate a scenario configuration before starting and shall reject missing network, controller, seed, duration, or demand inputs.
FR-02	The system shall start, pause, resume, and stop one controlled SUMO run while preserving an unambiguous run state.
FR-03	The simulation adapter shall be the sole owner of SUMO stepping and signal actuation.
FR-04	The perception worker shall process a selected recorded video and emit timestamped detections, tracks, source metadata, and processing status.
FR-05	The observation service shall aggregate valid vehicle observations into defined road, lane, or approach intervals without silently mixing sources.
FR-06	The operator shall be able to select fixed-time, rule-adaptive, or the configured RL controller before a run.
FR-07	Every signal action shall identify its intersection, requested and applied phase, effective simulation step, controller, and reason.
FR-08	The system shall enforce minimum green and configured yellow/all-red clearance before incompatible movements receive right of way.

ID	Requirement
FR-09	The emergency service shall validate ambulance origin and destination and return a clear no-route response if either is invalid or disconnected.
FR-10	The route service shall compute a valid remaining path using nonnegative time-based edge costs and shall record the cost snapshot.
FR-11	The rerouting service shall re-evaluate the remaining path after a closure, invalidated edge, or sustained cost improvement above the configured threshold.
FR-12	The corridor coordinator shall activate priority only for a bounded look-ahead set of intersections on the current ambulance route.
FR-13	The corridor coordinator shall detect passage or cancellation and restore the affected intersection to a safe normal-control state.
FR-14	A route revision shall cancel priority requests that no longer belong to the remaining path before activating new ones.
FR-15	The dashboard shall display the current source mode, simulation status, controller, signal state, traffic indicators, ambulance position, route, and corridor state.
FR-16	The dashboard shall present stale, invalid, disconnected, and fallback states visibly rather than continuing with apparently current data.
FR-17	The system shall record configuration, seed, component versions, periodic metrics, route revisions, signal events, failures, and termination reason for each run.
FR-18	The comparison service shall align selected runs by metric definition and warn when network, demand, duration, warm-up, or measurement versions are incompatible.
FR-19	The operator shall be able to export run configuration and metrics in a documented machine-readable format.
FR-20	On loss of the learned controller or invalid action, the control service shall fall back to a configured safe baseline and record the transition.

4.3 Quality Attributes

The quality model for Manzil follows the sample report's separation between broad software qualities and individually testable non-functional requirements. The qualities below explain why the requirements matter to this research prototype, while Section 4.4 assigns stable identifiers to the corresponding acceptance conditions. The distinction is important because a feature may execute without producing

credible research evidence. For example, a controller can return an action and still violate reliability if the applied phase, effective step, or fallback reason is lost. Similarly, a dashboard can appear responsive while showing stale observations. Manzil therefore treats usability, reliability, security, performance, maintainability, extensibility, and traceability as constraints on the complete workflow rather than as cosmetic objectives. Each quality is interpreted within a simulation and recorded-video setting; none is presented as evidence of certified public-road operation.

4.3.1 Usability

The interface is designed for a trained research operator who must configure a scenario, observe a run, initiate a simulated ambulance mission, and inspect retained evidence without switching among unrelated tools. Inputs use project terminology and display units, validation state, data source, and freshness beside the value that they qualify. Navigation preserves the selected run and mission context so that moving between the network overview, perception detail, emergency route, configuration, and analytics screens does not silently change the experiment. Status is not communicated by color alone: labels and symbols distinguish normal, stale, fallback, invalid, and disconnected states. Destructive or irreversible actions, such as stopping a run or replacing an active route version, require clear confirmation or are mediated by backend validation. These provisions do not claim that a formal usability study has been completed. They define observable interface behavior that can later be checked through task-based review using the proposed screens in Section 4.8.

4.3.2 Reliability and Integrity

Reliability concerns the preservation of valid state and evidence when a component is slow, unavailable, or produces an invalid request. One orchestrator and one SUMO adapter own simulation stepping and signal actuation, which prevents two services from advancing the same clock or issuing competing phase changes. Requested actions are stored separately from applied actions so the safety filter's intervention remains visible. A run configuration becomes immutable after start, route revisions are monotonically versioned, and corridor requests retain the route version that authorized them. Missing observations remain missing rather than being converted to zero, while stale observations are rejected or handled through a declared fallback. Terminal run states include completed, stopped, failed, and incomplete, each with a reason. Integrity therefore means that an unsuccessful experiment remains interpretable; it does not mean that every component is continuously available. Recovery and fallback behavior are specified explicitly in the use cases, risk register, and Chapter 6.

4.3.3 Security and Confidentiality

Manzil is not a public dispatch or traffic-control service, but its development data still require controlled handling. Recorded video must be licensed, public under suitable terms, or specifically authorized for team use. Raw video and model artifacts remain external files referenced through provenance and checksums rather than being copied indiscriminately into experiment exports. Machine credentials, private paths, and access tokens are excluded from run manifests. The proposed dashboard is an operator interface for a controlled laboratory environment and does not introduce public user registration merely because sample reports contain account modules. If authentication is later required for deployment, it must be added as a separate verified requirement rather than assumed in this deliverable. Exported research records should contain scenario and measurement identifiers but no unnecessary personal data. These boundaries provide proportionate confidentiality for the stated prototype while avoiding unsupported claims about production authorization, encryption infrastructure, or compliance with operational emergency-service policies.

4.3.4 Performance

Performance is defined relative to the chosen simulation step, control interval, video workload, and demonstration hardware. The control path must complete its decision before the configured deadline for the step in which the action becomes effective; otherwise the interval is marked late and a declared baseline or safe continuation is used. Video inference runs in a bounded worker so a slow model cannot block simulation stepping or an API request indefinitely. Dashboard delivery is a lossy view of authoritative state, allowing an inactive client to skip intermediate frames without delaying control. Route computation and comparison queries are profiled separately from simulation actuation because their acceptable latency depends on network size and update frequency. The repository contains no measured benchmark for the final hardware, detector, or road network, so exact thresholds remain design targets to be set after baseline profiling. This treatment prevents aspirational response times from being presented as achieved results and ensures that later measurements identify workload, hardware, and software versions.

4.3.5 Maintainability and Reusability

Maintainability is supported through typed, versioned contracts and replaceable implementations behind stable responsibilities. Fixed-time, rule-adaptive, and learned controllers share one request interface and therefore receive the same snapshot fields and legal phase constraints. The repository boundary allows early file-backed persistence to be replaced by PostgreSQL without changing the meaning of a run, while the perception boundary permits a detector or tracker to change only when its model version and

output semantics are declared. Configuration is stored in validated files rather than scattered constants, and public interfaces are documented with units, identifiers, timing semantics, and failure behavior. Tests target cost functions, route continuity, freshness rules, schema validation, and corridor invariants independently of the final dashboard. Reuse is limited to components whose assumptions remain valid; a model, road network, or phase plan is not treated as reusable merely because its file can be loaded. These rules reduce coupling while preserving the provenance required for research comparison.

4.3.6 Extensibility and Traceability

The design leaves controlled extension points for additional controllers, perception models, route-cost providers, scenarios, and visualizations. An extension must implement the existing contract or introduce an explicit schema version, and it must not acquire direct TraCI access or bypass the common safety filter. New measurements include a definition, unit, interval, and source run identifiers so previous and revised metrics cannot be combined silently. Traceability connects functional requirements to use cases, screens, architecture components, events, and planned tests through stable identifiers and cross-references, even though the standalone traceability matrix was removed at the supervisor's direction. Every exported comparison retains the run manifest, seed coverage, controller version, metric version, and failed-run status. This approach allows the prototype to grow without changing the interpretation of earlier evidence. Extensibility is therefore not an unrestricted plugin claim; it is the ability to replace or add a component while declaring the effect on control ownership, data contracts, and reproducibility.

4.4 Non-Functional Requirements

Functional behavior alone would permit a demonstration that appears successful but cannot be reproduced or audited. Table 4.2 therefore states the quality attributes that constrain every feature group. Reproducibility and data integrity require a run to preserve the exact scenario, version, seed, and metric definition used to generate a result. Safety requirements apply within simulation and prevent controllers from bypassing clearance or ownership rules. Freshness and isolation ensure that a stale perception result or slow dashboard cannot silently influence a later simulation step. Responsiveness is intentionally expressed as a profiled design target because the repository contains no defensible hardware benchmark from which to invent a deadline. Privacy is limited to the handling of licensed or team-authorized video and does not imply a production governance framework. These non-functional requirements also define how negative outcomes are handled: an unavailable controller, disconnected simulator, or incomplete record must remain visible rather than being removed from the evaluation set without explanation.

Table 4.2: Non-functional requirements and design targets

ID	Attribute	Requirement or target
NFR-01	Reproducibility	A run shall store network and demand identifiers, seed, controller configuration, software/model versions, warm-up, duration, and metric schema.
NFR-02	Safety in simulation	Incompatible green movements shall never be applied; every ordinary and emergency transition shall pass the safety filter and use the configured clearance sequence.
NFR-03	Auditability	Applied decisions shall be distinguishable from requests, and their simulation timestamps and reasons shall be retained.
NFR-04	Freshness	Each observation shall carry produced-at and effective simulation time. Stale thresholds are configuration values and stale data shall not silently drive control.
NFR-05	Responsiveness	The control action for step t should be available before its configured decision deadline. The exact deadline is a design target to be set after baseline profiling, not an achieved value.
NFR-06	Isolation	Background video inference and dashboard clients shall not advance SUMO or directly actuate signals.
NFR-07	Usability	A trained operator should be able to configure the core demonstration from one experiment screen, with units and validation messages visible.
NFR-08	Portability	Services should be runnable from versioned environments; containerization is proposed after local baseline integration is stable.
NFR-09	Maintainability	Component interfaces shall use typed, versioned data contracts and controller implementations shall share one interface.
NFR-10	Data integrity	A completed run configuration shall be immutable; derived summaries shall retain their metric version and source run identifiers.

ID	Attribute	Requirement or target
NFR-11	Privacy	Video shall be limited to licensed or team-authorized material, stored only as needed, and excluded from exports unless explicitly requested.
NFR-12	Availability	The prototype shall fail visibly and recover to a safe stopped or baseline-controlled simulation state; no public-service availability is claimed.

4.5 Assumptions

SUMO and TraCI are required for controlled traffic experiments [15, 20]. A selected network must contain legal lane connections and signal plans after import and manual cleanup, and representative routes must be validated before it is used for comparison. Video sources require sufficient visibility, a documented license or team authorization, and annotations appropriate to the selected measure. The simulation assumes one clock owner, nonnegative route costs, and emergency endpoints on ambulance-accessible edges. It also assumes that configured phase transitions reflect the validated simulation network; they are not certified timings for physical signal hardware. The dashboard is an operator and evaluator tool rather than an autonomous public dispatch system, and a displayed route does not constitute operational navigation advice. When an assumption cannot be met, the affected workflow must stop or disclose a bounded fallback. It must not silently substitute a different network, data source, time unit, or observation mode because that would make later comparisons irreproducible.

React and FastAPI are fixed by the approved proposal for the interface and application service boundary [21, 22]. Python is implied by the planned perception, reinforcement-learning, and TraCI ecosystem. PostgreSQL, a WebSocket transport, Docker, and specific chart or map libraries are proposed dependencies and remain replaceable until implementation confirms them. These assumptions are deliberately narrower than a production deployment specification. Manzil is evaluated as a controlled prototype on prepared networks, not as an interface to public signal controllers or ambulance dispatch infrastructure. A selected video source must have suitable permission and sufficient visibility for the intended annotation task, while the simulation network must be checked for lane connectivity, valid traffic-light programs, and ambulance-accessible routes. If any assumption fails, the run must be rejected or marked with an explicit limitation instead of substituting an undocumented configuration. This treatment keeps dependencies traceable and prevents a convenient library choice from becoming an unsupported architectural claim.

4.6 Use Cases

The primary actor is a research operator who configures experiments, observes their progress, initiates a simulated emergency mission, and exports evidence. SUMO, recorded video, and the experiment store support those actions but do not independently choose research parameters. Evaluators consume dashboard views and exported records without receiving control privileges. Figure 4.1 presents this boundary and connects the operator to eight goal-oriented interactions. Configuration and simulation appear separately because validation must finish before the single clock owner starts a run. Dispatch, corridor monitoring, and rerouting are also distinct so that a route revision cannot be treated as an invisible side effect. Tables 4.3–4.10 then specify a summary, conditions, special requirements, the basic actor–system exchange, and alternative behavior for each interaction. Collectively, they test more than the successful interface path: invalid endpoints, stale streams, no-route cases, controller failure, incompatible comparisons, and termination reasons must all produce observable outcomes. The use cases therefore define acceptance evidence as well as operator interaction.

Table 4.3: UC-01: Configure a Scenario

Name		UC-01: Configure a Scenario	
Actors		Research operator	
Summary		The operator prepares a validated and versioned experiment configuration.	
Pre-Conditions		Network, demand, controller, and source descriptors are available.	
Post-Conditions		A valid configuration is stored; no run has started.	
Special Requirements		Identifiers, durations, seeds, and controller compatibility must be validated.	
Basic Flow			
Actor Action		System Response	
1	Opens experiment configuration.	2	Displays available networks, demand profiles, controllers, and source modes.
3	Selects settings and submits.	4	Validates the fields and stores the versioned configuration.
Alternative Flow			
3	Submits missing or incompatible values.	4-A	Identifies each invalid field and does not create a runnable configuration.

4.7 Hardware and Software Requirements

The development environment must support two different workloads: repeated model or controller experiments and a prepared, repeatable demonstration. The distinction avoids presenting a high-end training machine as a mandatory deployment requirement. Hardware values in this section are capability

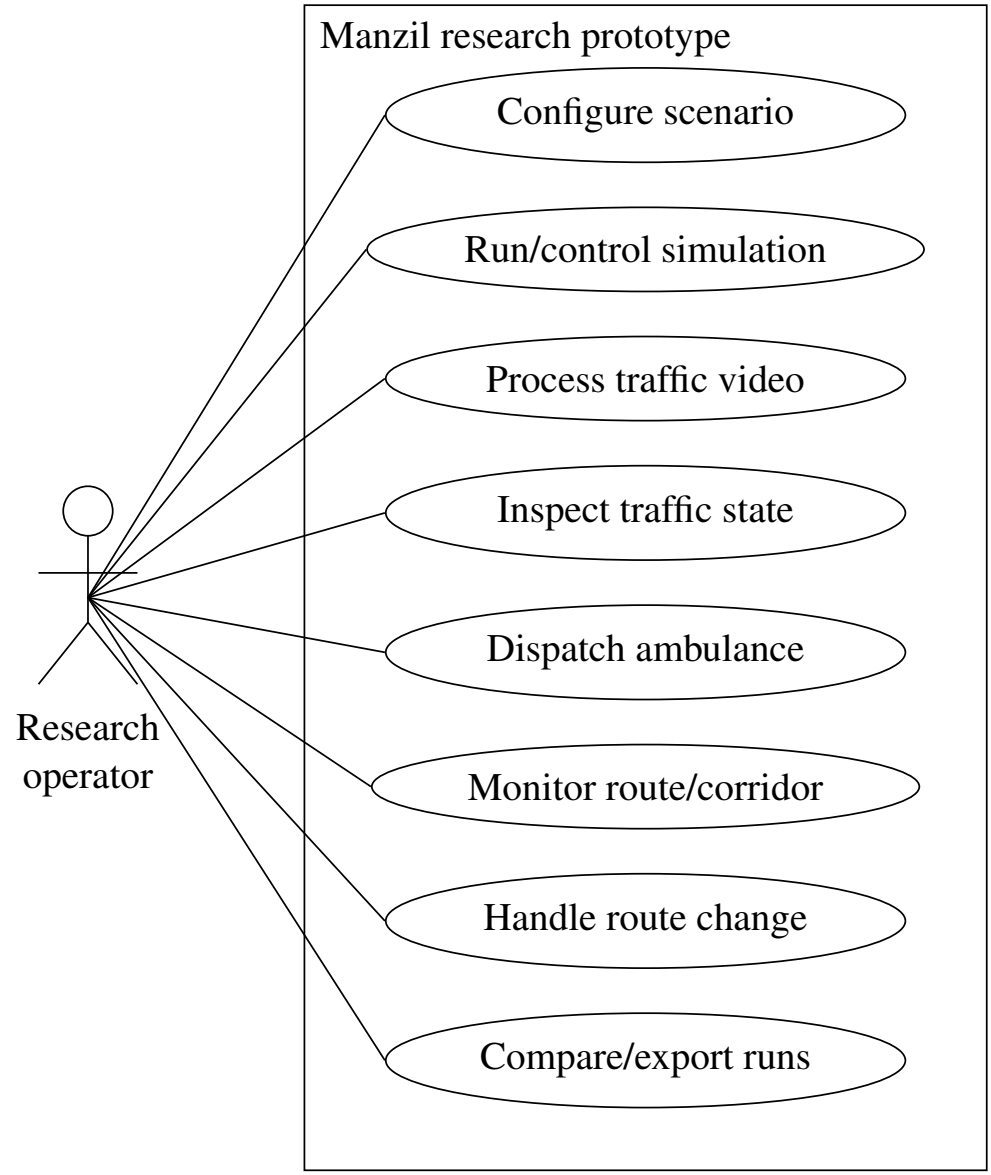


Figure 4.1: Proposed operator use-case diagram

Table 4.4: UC-02: Run and Control a Simulation

Name		UC-02: Run and Control a Simulation	
Actors		Research operator	
Summary		The operator starts and monitors a configured SUMO experiment.	
Pre-Conditions		UC-01 is valid and the SUMO adapter is available.	
Post-Conditions		The run is completed, stopped, or failed with an explicit terminal reason.	
Special Requirements		Only the orchestrator may advance simulation time; illegal actions require fallback.	
Basic Flow			
Actor Action		System Response	
1	Selects Start for a valid configuration.	2	Creates the run, launches SUMO, and completes warm-up.
3	Monitors, pauses, resumes, or stops the run.	4	Steps the simulation, applies lawful actions, records evidence, and terminates safely.
Alternative Flow			
1	Starts an unavailable or invalid setup.	2-A	Rejects the start or records launch/TraCI failure while retaining diagnostic logs.

Table 4.5: UC-03: Process Traffic Video

Name		UC-03: Process Traffic Video	
Actors		Research operator	
Summary		The operator creates a video-processing job that produces traceable traffic observations.	
Pre-Conditions		A licensed video source and model configuration are available.	
Post-Conditions		Stored observations identify the source, model, interval, units, confidence, and job status.	
Special Requirements		Uncalibrated video may report proxies but not physical queue length or speed.	
Basic Flow			
Actor Action		System Response	
1	Selects a video source and model configuration.	2	Validates provenance, format, model, and optional calibration.
3	Starts processing.	4	Detects, tracks, assigns regions, aggregates observations, and records job progress.
Alternative Flow			
3	Submits unreadable media or processing fails.	4-A	Reports the failure and preserves completed evidence without inventing physical measures.

Table 4.6: UC-04: Inspect Traffic State

Name		UC-04: Inspect Traffic State	
Actors		Research operator	
Summary		The operator reviews current or recorded traffic state without changing control decisions.	
Pre-Conditions		A simulation run or perception job has emitted observations.	
Post-Conditions		The displayed state remains traceable to its source and no decision is changed.	
Special Requirements		Source mode, units, freshness, confidence, and invalid values must remain visible.	
Basic Flow			
Actor Action		System Response	
1	Opens the overview or an intersection detail.	2	Displays source, freshness, counts, queue values, phases, controller, and warnings.
3	Selects another time or intersection.	4	Retrieves the corresponding traceable snapshot.
Alternative Flow			
1	Opens a disconnected or stale stream.	2-A	Marks the state stale or unavailable and does not present the last value as current.

Table 4.7: UC-05: Dispatch an Ambulance

Name		UC-05: Dispatch an Ambulance	
Actors		Research operator	
Summary		The operator creates one simulated emergency mission between valid network positions.	
Pre-Conditions		An emergency-enabled scenario is running and no core mission is active.	
Post-Conditions		A mission is active with a valid route, or the request is rejected cleanly.	
Special Requirements		Origin and destination must be legal vehicle edges and corridor activation must be atomic.	
Basic Flow			
Actor Action		System Response	
1	Selects origin and destination and requests dispatch.	2	Validates the endpoints and computes a time-based legal route.
3	Confirms the valid mission.	4	Spawns the ambulance, publishes the route version, and requests bounded look-ahead.
Alternative Flow			
1	Submits invalid endpoints, no-route input, or a duplicate mission.	2-A	Explains the rejection and activates no partial corridor.

Table 4.8: UC-06: Monitor Route and Corridor

Name		UC-06: Monitor Route and Corridor	
Actors		Research operator	
Summary		The operator follows ambulance movement, route versions, and intersection priority states.	
Pre-Conditions		An emergency mission is active.	
Post-Conditions		Mission progress and every priority transition are recorded.	
Special Requirements		Priority must obey legal transition, expiry, passage, timeout, and recovery rules.	
Basic Flow			
Actor Action		System Response	
1	Opens the active emergency mission.	2	Displays position, remaining route, current route version, and corridor states.
3	Continues monitoring as the ambulance advances.	4	Requests safe priority, detects passage, releases ownership, and records recovery.
Alternative Flow			
3	A position becomes stale, a request is unsafe, or timeout occurs.	4-A	Cancels affected requests, applies fallback or recovery, and displays an alert.

Table 4.9: UC-07: Handle a Blockage or Route Change

Name		UC-07: Handle a Blockage or Route Change	
Actors		Research operator	
Summary		The system evaluates rerouting from the ambulance's valid current position.	
Pre-Conditions		An active mission exists and a closure or sustained cost change is detected.	
Post-Conditions		Exactly one route version remains current and every decision retains its cause and cost snapshot.	
Special Requirements		Feasibility, improvement threshold, persistence, cooldown, and obsolete-request cancellation apply.	
Basic Flow			
Actor Action		System Response	
1	Reviews the blockage or reroute notification.	2	Computes and validates a candidate route under one cost snapshot.
3	Continues the mission after the decision.	4	Publishes an accepted version atomically or retains the current route with a reason.
Alternative Flow			
3	No feasible route exists.	4-A	Cancels future priority, marks the mission blocked, and records the graph snapshot.

Table 4.10: UC-08: Compare and Export Runs

Name		UC-08: Compare and Export Runs	
Actors		Research operator	
Summary		The operator compares eligible experiments and exports their evidence and manifests.	
Pre-Conditions		At least two completed or failed run records exist.	
Post-Conditions		The comparison and export identify source run IDs, metric definitions, and schema version.	
Special Requirements		Network, demand, warm-up, duration, seed coverage, and metric versions must be disclosed.	
Basic Flow			
Actor Action		System Response	
1	Selects runs and a metric.	2	Checks comparability and displays distributions, variability, failures, and configuration differences.
3	Requests an export.	4	Packages selected records with identifiers, units, definitions, and manifests.
Alternative Flow			
1	Selects incompatible runs.	2-A	Warns or refuses direct comparison and never silently omits failed runs.

statements because the selected Lahore network, final detector, training budget, and demonstration machine have not yet been fixed. Software choices are separated from hardware so that a library can be versioned or replaced without changing the resource description. Tables 4.11 and 4.12 identify the responsibilities and confirmation points that must be resolved before implementation evidence is reported. The demonstration must use the same network, controller configuration, metric schema, and software versions as the retained evaluation record even if it runs fewer scenarios. Measured CPU, GPU, memory, storage, and response-time values will replace the current planning statements only after the representative workload has been profiled.

4.7.1 Hardware Requirements

Table 4.11 separates development and training resources from the minimum capabilities needed to reproduce a prepared demonstration. A multi-core processor is required for SUMO, service processes, preprocessing, and analysis. A CUDA-capable GPU is desirable for detector experiments and reinforcement-learning training, but its model and memory capacity remain unconfirmed; it is not required for every dashboard or SUMO-only workflow. Storage must accommodate approved video, virtual environments, checkpoints, network files, traces, and repeated run records while preserving checksums and versions. The demonstration system needs enough memory and compute to run the selected network, API, dashboard, and prepared policies without unbounded queues. If live video inference cannot meet the profiled budget, recorded inference output may be demonstrated as an explicitly labelled fallback. These condi-

tions keep resource claims honest and allow the team to scale the network or model before committing to a numerical minimum.

Table 4.11: Hardware resources for development and demonstration

Resource	Development and training	Prepared demonstration
Processor	Multi-core CPU for SUMO, preprocessing, services, and repeated analysis	CPU able to run the selected SUMO network, API, and dashboard at the configured step rate
Graphics processor	CUDA-capable GPU desirable for detector and RL experiments; exact model and memory are unconfirmed	Required only when live inference is included; otherwise prepared outputs or CPU-safe models may be used and labelled
Memory	Capacity for concurrent simulator, worker, services, model, and analysis processes; amount to be profiled	Capacity for the selected scenario and services without unbounded swapping or queue growth
Storage	Approved video, environments, checkpoints, maps, traces, exports, and backups	Versioned scenario, required models, application build, and demonstration records
Display/network	Development display and local networking for dashboard and service testing	Display suitable for the dashboard; local connectivity only unless a reviewed deployment requires more

4.7.2 Software Requirements

Table 4.12 follows the sample report's responsibility-oriented presentation: each technology is tied to a task and to a configuration item that must be recorded. SUMO and TraCI provide the authoritative simulation environment and control interface. Python supports perception, routing, control, experiment orchestration, and analysis, while React and FastAPI form the proposed interface boundary from the approved proposal. A detector, tracker, and reinforcement-learning library will be selected and versioned during implementation; the report does not claim a final checkpoint or tracker configuration. OpenStreetMap tools and SUMO NetEdit support network preparation, with a documented synthetic network retained as fallback. PostgreSQL remains proposed for structured persistence, but a file-backed repository may be used during integration if it preserves the same schema semantics. Git and environment lock files identify the software state used for every retained experiment.

4.8 Graphical User Interface

The interface specification follows the sample report by defining the screen inventory and navigation before presenting individual wireframes. The five views in Table 4.13 are proposed screens rather than screenshots of implemented software. They validate information hierarchy, source disclosure, error visibility, and the actions needed for a controlled demonstration without fabricating implementation

Table 4.12: Software technologies and configuration responsibilities

Technology or resource	Responsibility	Configuration status
Python	Orchestration, perception, control, routing, metrics, and analysis	Interpreter and package lock recorded per experiment release
SUMO and TraCI	Microscopic traffic simulation, state access, route application, and signal actuation	Version and validated network package recorded
React	Operator dashboard and evidence views	Approved proposal choice; component and build versions to be recorded
FastAPI	Command, validation, query, and streaming boundary	Approved proposal choice; API schema versioned
Detector, tracker, and RL libraries	Vehicle observation and learned-controller experiments	Final frameworks, checkpoints, and hyperparameters remain to be confirmed
OpenStreetMap tools and SUMO NetEdit	Network extraction, cleanup, connection, route, and signal validation	Exact Lahore extract remains to be selected; synthetic fallback retained
Repository and persistence tools	Version control, manifests, structured records, and artifacts	Git required; PostgreSQL proposed behind a replaceable repository interface

progress. The configuration view establishes an immutable run, the overview supports monitoring, the perception view explains one intersection, the emergency view exposes route and corridor state, and the analytics view supports comparison after completion. Figure 4.2 shows the permitted navigation among these views. Monitoring pages do not acquire control authority: every command still passes through the validated API boundary. This separation lets a reader understand both the interface flow and the evidence available at each stage while preserving the project status stated elsewhere in the report.

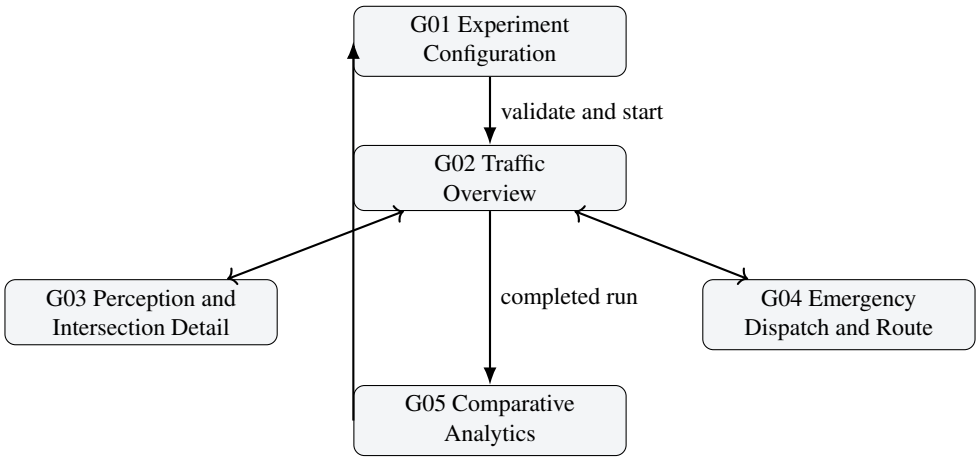


Figure 4.2: Proposed primary navigation flow between Manzil screens

The overview and perception wireframes provide complementary levels of detail. Figure 4.3 gives the operator a network-level view in which the selected run, controller, source mode, ambulance route,

Table 4.13: Proposed graphical user interface screen inventory

ID	Screen	Primary user	Purpose
G01	Experiment Configuration	Operator	Validate and start a versioned scenario and controller configuration.
G02	Traffic Overview	Operator and evaluator	Monitor network state, warnings, controller source, and active mission.
G03	Perception and Intersection Detail	Operator and evaluator	Inspect observation provenance, freshness, confidence, and approach-level measures.
G04	Emergency Dispatch and Route	Operator	Create a mission and monitor route versions and corridor transitions.
G05	Comparative Analytics	Operator and evaluator	Check comparability, inspect metrics and failed runs, and export evidence.

corridor lifecycle, and warnings remain visible together. This arrangement prevents a plausible map animation from hiding stale data or a fallback controller. Figure 4.4 narrows the view to one intersection and pairs a processed-video preview with model provenance, freshness, regional measures, confidence, and invalid-state indicators. The calibration field is explicit because pixel-derived queue proxies must not be mislabeled as physical queue length. Both designs use color only as a supporting cue; labels retain the meaning required for accessibility and printed evidence. The reader can therefore distinguish authoritative values, estimates, and warnings without relying on animation.

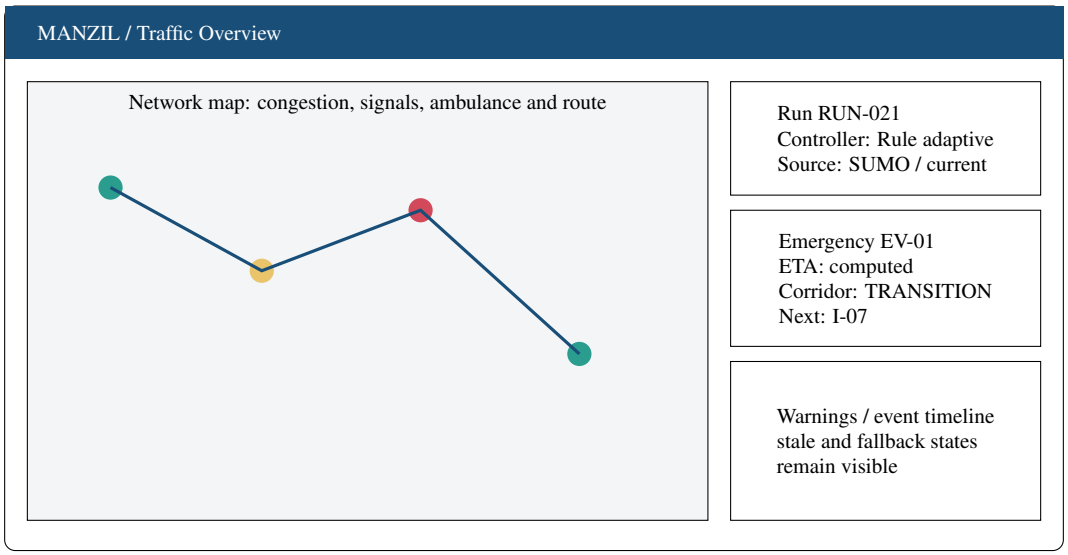


Figure 4.3: G02 traffic-overview screen wireframe

Emergency operation and experiment setup require different decisions, so they are separated in Figures

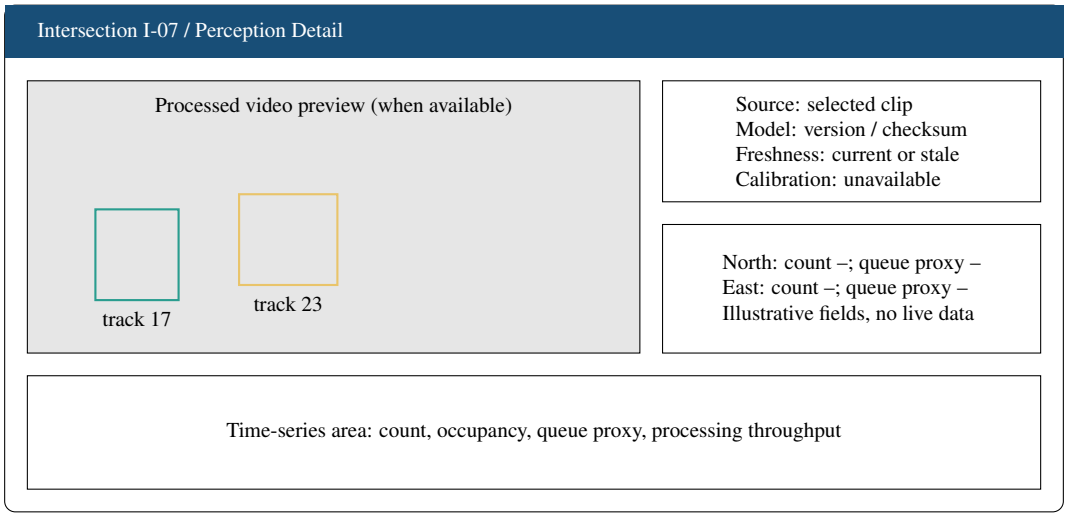


Figure 4.4: G03 perception and intersection-detail screen wireframe

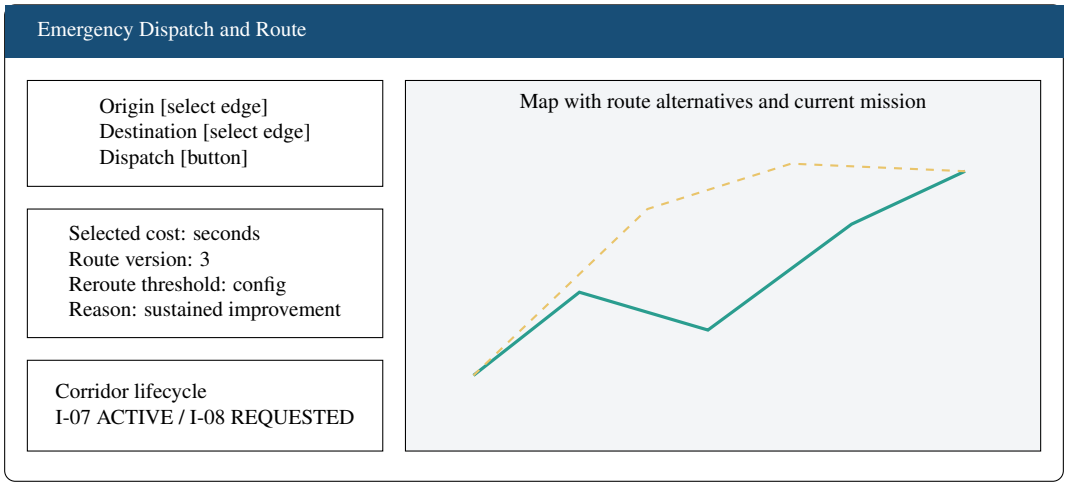


Figure 4.5: G04 emergency-dispatch and route screen wireframe

4.5 and 4.6. The dispatch view shows validated endpoints, the current route version, its time-based cost, the reason for a revision, and the state of intersections in the bounded look-ahead corridor. It also permits a candidate route to be shown without implying that every alternative has been activated. The configuration view collects the network, demand, controller, seed, duration, incident, mission, and metric schema before a run begins. Its validation panel exposes unresolved identifiers, illegal phase plans, absent calibration, or checkpoint mismatch, and the start action remains unavailable until mandatory errors are resolved. The two wireframes therefore support auditability at different stages: one explains a live mission decision, while the other preserves the conditions under which that decision can later be compared. Neither view grants direct signal control, bypasses backend validation, or changes an active run's immutable configuration.

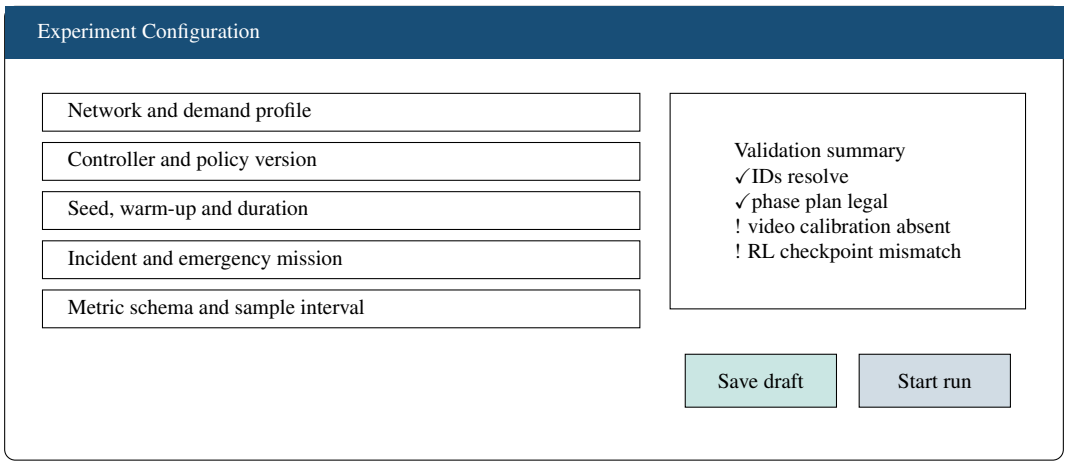


Figure 4.6: G01 experiment-configuration screen wireframe

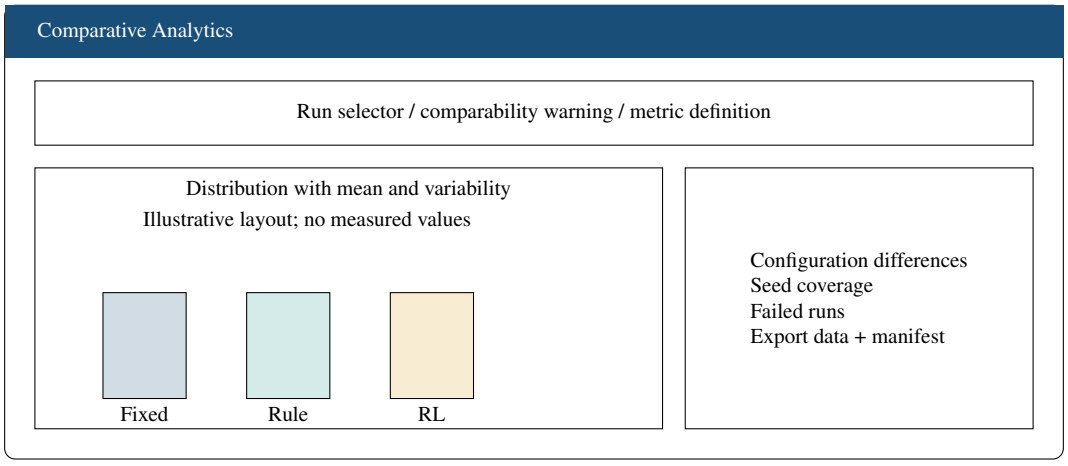


Figure 4.7: G05 comparative-analytics screen wireframe

Figure 4.7 represents the final evidence-review workflow. Run selection is paired with a comparability warning and metric definition so that two visually adjacent bars are not assumed to represent matched experiments. The chart area is schematic and contains no claimed result; an implemented view would present the chosen distribution, central tendency, variability, and sample count from retained run records.

The adjoining panel exposes configuration differences, seed coverage, failed runs, and export controls. This design responds directly to the methodology requirement that failed or incompatible runs must not disappear from a favorable summary. It also keeps measurement units and metric versions close to the visualization, reducing the risk that a screenshot is detached from its experimental meaning. Together, the five wireframes define the minimum information architecture for the prototype while leaving visual polish and specific chart libraries open to implementation. Exported graphics must retain the same metric and run identifiers.

4.9 Database Design

4.9.1 Entity Relationship Diagram

Persistent data is reserved for configuration, observations, decisions, missions, and outcomes that require audit or comparison. The live road graph, TraCI subscription, and frame buffers remain runtime state, while raw video is retained as a checksummed external artifact. Following the notation used in the reference report, Figure 4.8 presents the complete conceptual model with blue entity rectangles, green attribute ovals, amber relationship diamonds, and cardinalities beside relationship lines. The dashed regions separate experiment configuration and evidence, observation and control records, and emergency routing and priority. Figures 4.9, 4.10, and 4.11 enlarge those regions so keys, attributes, and named relationships remain readable at normal print size. Primary keys are underlined, foreign keys are marked FK, and optional ownership is expressed explicitly. The diagrams describe a proposed logical schema and do not claim that a production database has already been populated.

Figure 4.9 enlarges the experiment-configuration region of the overview. A Scenario defines immutable network and demand versions and may be reused by several ExperimentRun records. Each run fixes its seed, controller version, status, and configuration at start, then owns its MetricSample and Artifact records. The records relationship is deliberately named because metrics and exported files are evidence produced under one run rather than free-standing data. Metric samples retain a definition version, interval, value, and unit, while artifacts retain a type, URI, checksum, and provenance. These associations prevent results from different seeds or controller configurations from being combined without their originating run. They also permit an evaluator to trace a displayed chart or exported file back to the configuration that produced it.

Figure 4.10 separates the two permitted observation owners and connects observations to control evidence. Simulation observations reference an ExperimentRun; offline video observations reference a VideoJob. Exactly one owner is present for each observation, so a record cannot be presented simultaneously as simulator truth and video-derived evidence. The Observation entity carries source mode, entity

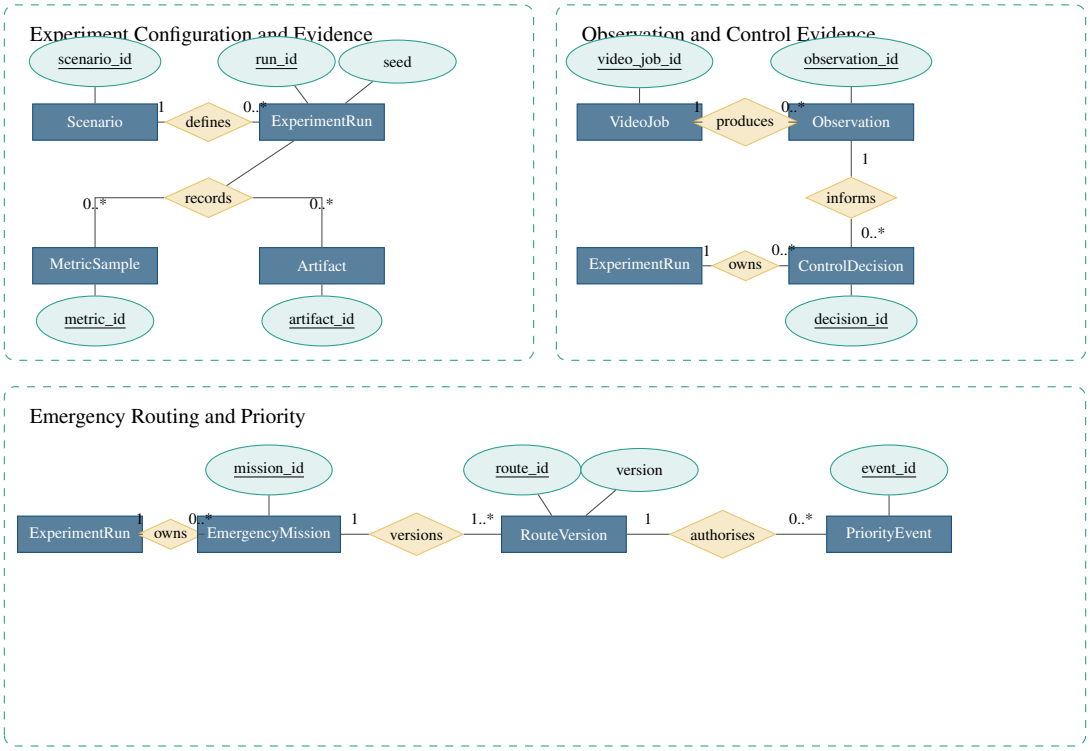


Figure 4.8: Complete grouped conceptual ER model using entity, attribute, and relationship symbols

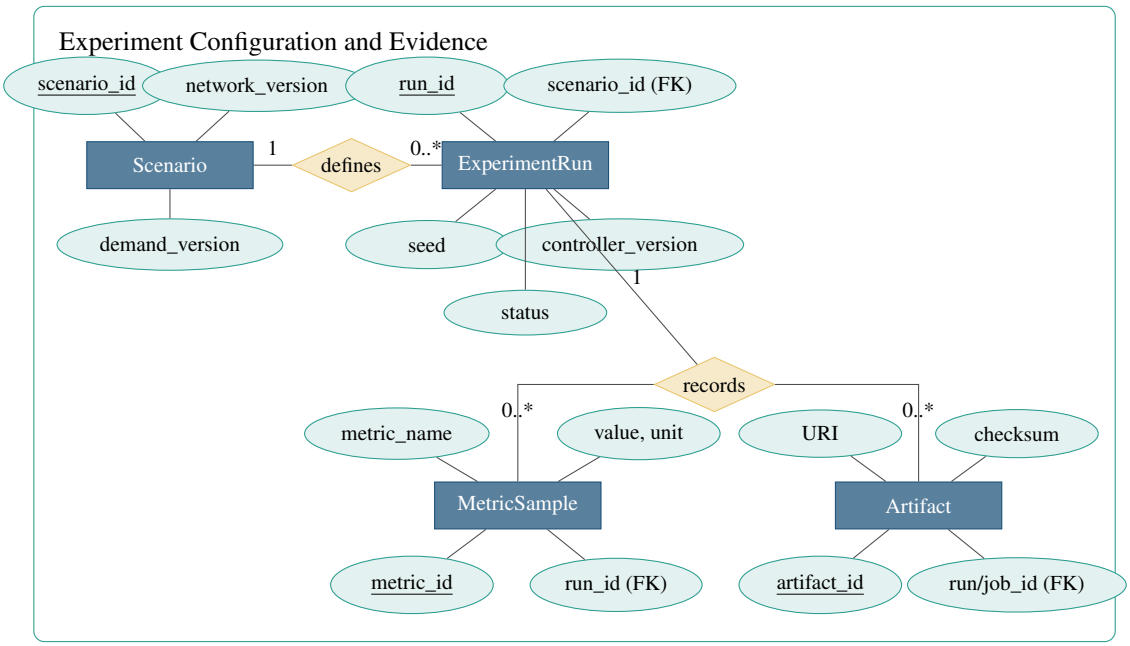


Figure 4.9: Experiment scenarios, runs, metrics, and artifacts with conceptual attributes

identifier, effective and validity times, values, units, and confidence. A ControlDecision refers to the run and to the observation context used by the controller, while preserving requested and applied phases and any safety reason. The named informs relationship expresses causal use rather than simple co-location. This structure supports stale-data checks and permits the evaluator to distinguish a controller error from an unavailable observation, rejected action, or safety transformation.

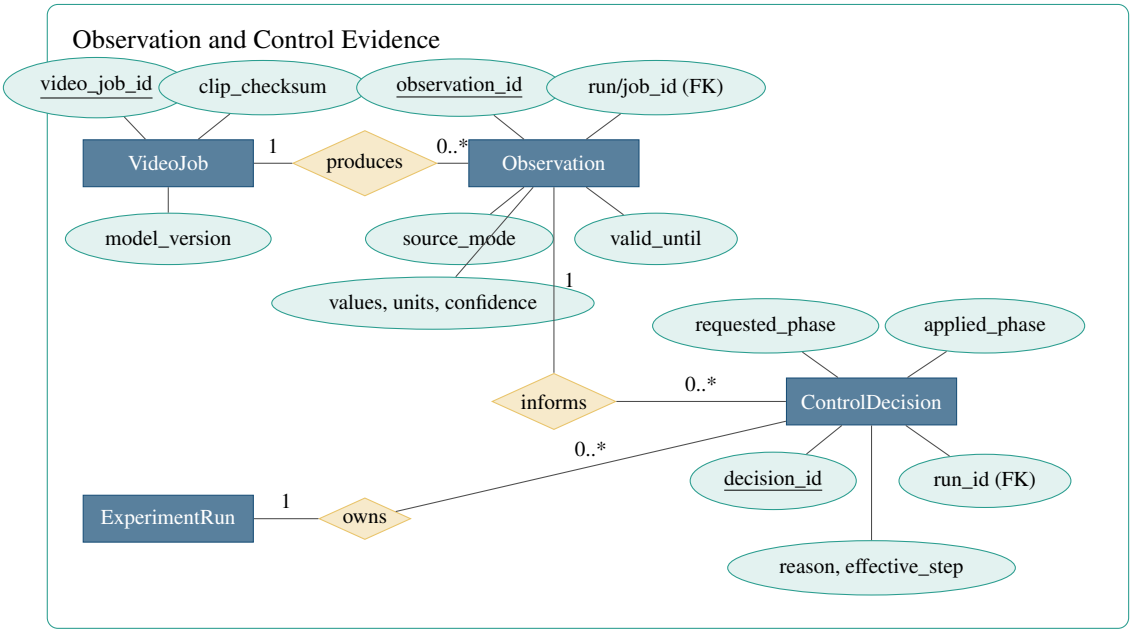


Figure 4.10: Observation ownership and control-decision evidence with conceptual attributes

Figure 4.11 enlarges the emergency-control chain. A run can own several mission attempts over its lifetime, although the core scope permits only one active mission at a time. Each accepted dispatch creates an EmergencyMission with vehicle, origin, destination, and status. An accepted route or reroute creates a new immutable RouteVersion rather than overwriting the previous path. The versions relationship is one-to-many and the version number increases within its mission. PriorityEvent records then reference the exact route version that authorised a request, safe transition, active-priority interval, passage, cancellation, or recovery state. This structure allows obsolete priority work to be cancelled after rerouting and later explains which path justified every intersection action. The ER notation therefore reflects the same causal ownership enforced by the corridor state machine.

4.9.2 Data Dictionary

Table 4.14 defines the principal identifiers and integrity constraints represented in the ER diagrams. It adds the provenance and validity rules that cannot be communicated by cardinality alone, including immutable run configuration, mandatory observation source and units, monotonically increasing route versions, and metric-version compatibility. The dictionary also distinguishes large external artifacts from relational records so videos, models, and logs can remain checksummed files without becoming database

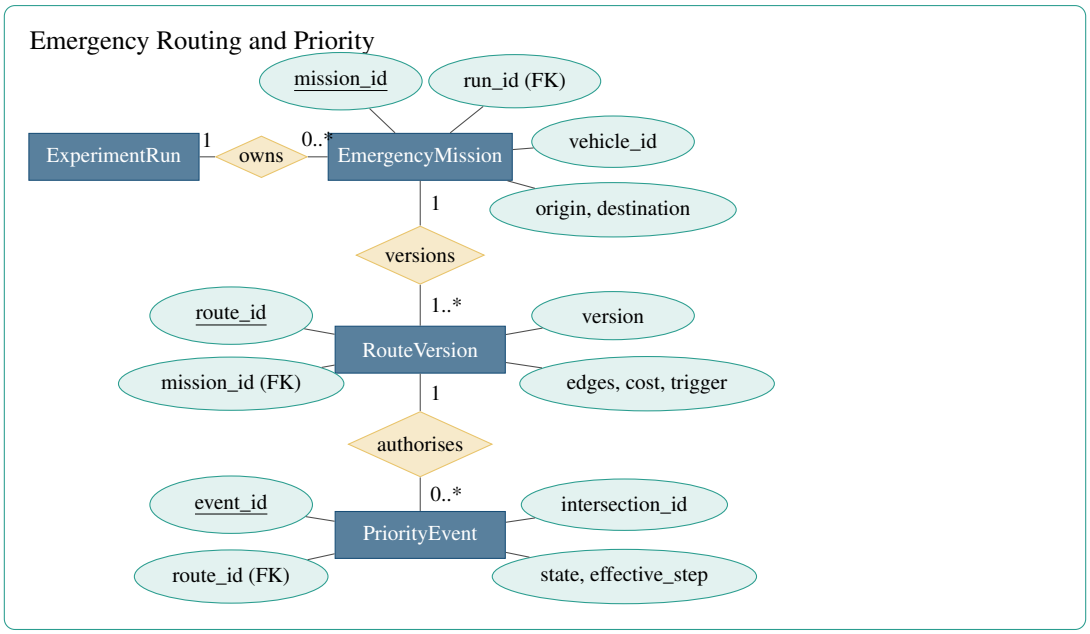


Figure 4.11: Emergency missions, route versions, and priority events with conceptual attributes

blobs. Together, the diagram and dictionary establish the proposed storage contract for implementation and evaluation.

Table 4.14: Data dictionary

Entity	Key fields	Integrity rule
Scenario	scenario_id, network_version, demand_version	References immutable network and demand manifests.
Run	run_id, scenario_id, seed, controller_version, status	Configuration becomes immutable after start; terminal reason required.
VideoJob	job_id, clip_checksum, model_version, status	Clip provenance and model version become immutable after processing begins.
Observation	run_id or job_id, source_mode, entity_id, observed_at, valid_until, values, units	Exactly one owner is present; source and units are mandatory and validity cannot be inferred.
ControlDecision	run_id, intersection_id, requested_phase, applied_phase, effective_step, reason	Requested and applied actions are separate fields.

Entity	Key fields	Integrity rule
Mission	mission_id, run_id, vehicle_id, origin, destination, status	Only one active core mission per run.
RouteVersion	mission_id, version, edge_list, cost_seconds, trigger, created_at	Version increases monotonically; path begins at valid current position.
PriorityEvent	mission_id, route_version, intersection_id, state, requested/applied time	References the route version that justified priority.
MetricSample	run_id, metric_name, metric_version, value, unit, interval	Comparisons require matching metric name, version, and unit.
Artifact	run/job_id, type, URI, checksum, license/provenance	Content is addressable and provenance is retained.

4.10 Risk Analysis

Table 4.15 records technical and research risks that could invalidate the prototype or its conclusions. Likelihood and impact are qualitative planning judgments rather than measured probabilities. The highest-consequence risks concern invalid signal phases, irreproducible results, unsuitable map imports, and evidence that cannot be legally or reliably used. Each mitigation is paired with a bounded fallback so that the team can preserve an honest deliverable if a preferred component fails. For example, a documented synthetic network is preferable to an unverified real-map extract, and a negative reinforcement-learning result is preferable to tuning on the final test scenarios. The register also treats timing drift, route oscillation, occlusion, and reward imbalance as system-level concerns rather than isolated coding defects. These risks are revisited through invariant tests, scenario checks, provenance records, and the controlled comparisons defined in Chapter 5; unresolved items must remain visible in the final limitations rather than being converted into unsupported performance claims.

Table 4.15: Risk register

Risk	Rating	Mitigation	Fallback
Map import lacks valid signals/turns	Medium likelihood; high impact	Inspect and clean a small extract early; validate routes and phases	Use a documented synthetic grid
Local video lacks labels or permission	Medium likelihood; high impact	Use licensed public sequences and record provenance	Evaluate public data only; state domain gap
Occlusion causes unstable counts	High likelihood; medium impact	Use region rules, temporal aggregation, and tracker comparison	Limit claims to labelled conditions
RL fails to converge or generalize	Medium likelihood; high impact	Use stable baselines, bounded actions, and training-only tuning	Report a negative or partial finding
Reward improves one metric unfairly	Medium likelihood; high impact	Evaluate multiple metrics and enforce starvation constraints	Reject a policy that violates fairness checks
Rerouting oscillates	Medium likelihood; medium impact	Apply a benefit threshold, persistence, cooldown, and closure exception	Retain the current feasible route
Preemption leaves unsafe phase	Low likelihood; very high impact	Use one state machine and test transition invariants	Cancel and enter a safe recovery path
Service timing drifts from simulation	Medium likelihood; high impact	Use one step owner, simulation timestamps, and bounded queues	Run synchronously without live perception
Insufficient compute	Medium likelihood; medium impact	Profile early; reduce network/model size and train offline	Demonstrate recorded inference and compact scenarios
Results are not reproducible	Medium likelihood; high impact	Retain immutable manifests, seeds, versioned metrics, and exports	Exclude untraceable runs

4.11 Conclusion

The specification centers on traceability and failure visibility rather than on a collection of loosely connected features. It requires one simulation-clock owner, legal signal transitions, stable route revision, explicit observation freshness, immutable run configuration, and comparisons that cannot hide incompatible settings or failed trials. Functional requirements define what the operator and services must observe, while non-functional requirements constrain timing, ownership, privacy, auditability, and recovery. The use cases exercise both successful and failed paths, the wireframes show how critical state and provenance remain visible, and the data model preserves the route and decision history needed for later analysis. The risk controls connect those requirements to bounded fallbacks and planned verification. Chapter 5 converts the specification into a staged research protocol, and Chapter 6 assigns each mutable resource and interface to a clear architectural owner. Implementation status will continue to be reported separately so that a specified feature is never presented as completed evidence.

Chapter 5 Proposed Approach and Methodology

This chapter defines how Manzil will be constructed and evaluated so that its objectives can be assessed with controlled evidence rather than a favorable demonstration run. The method begins by freezing definitions, exclusions, baselines, and data partitions before the final comparison. It then validates the network, observations, controllers, routing, and corridor lifecycle in stages, because a result from an integrated run is not interpretable when an underlying measurement or safety transition is still uncertain. Figure 5.1 summarizes this sequence from preparation to reporting. Each stage produces an artifact or acceptance check that is retained with the experiment manifest. The protocol also separates detector evaluation from signal-control evaluation: recorded video supplies annotated evidence for perception, while SUMO supplies controlled vehicle and signal state for the controller, route, and emergency experiments. This separation prevents estimated video quantities from being passed off as exact network state and allows an optional observation-noise study only after both baselines are stable.

5.1 Overall Research Workflow

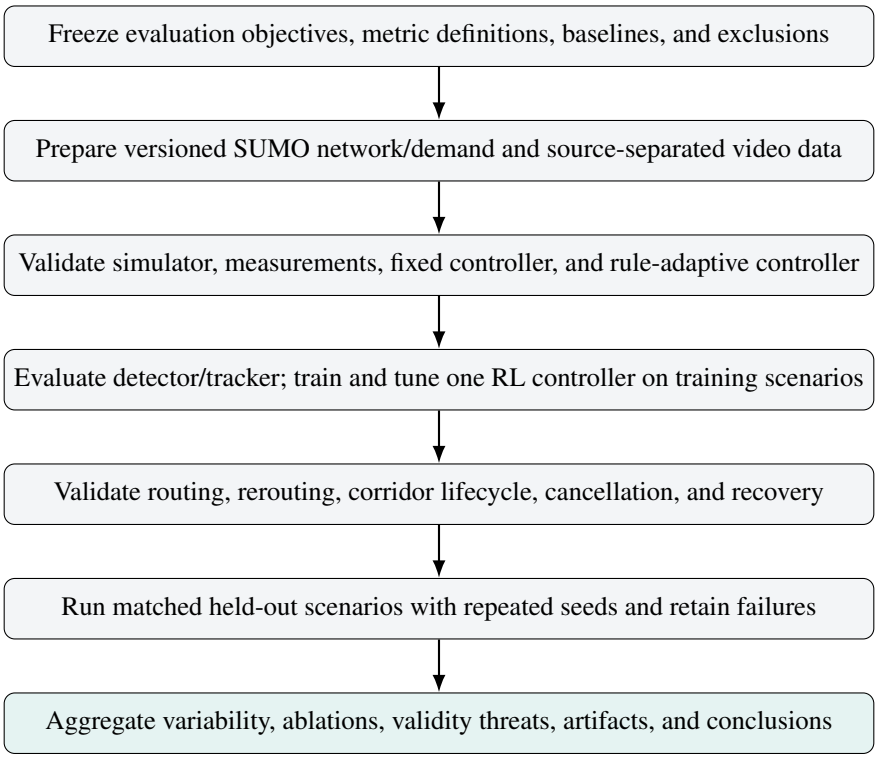


Figure 5.1: Proposed research and evaluation workflow

The workflow in Figure 5.1 is ordered to prevent later components from masking earlier faults. Evaluation objectives and metric definitions are fixed before data preparation; the network and demand are validated before controllers are compared; and ordinary traffic control is established before emergency

routing and priority are enabled. Two input modes are retained throughout this process. In video mode, annotations provide the reference for perception metrics, and no claim is made that a selected clip determines the full state of the simulated network. In simulation mode, SUMO supplies lane, vehicle, signal, and route state for controlled experiments. Both modes use the same typed traffic-state concepts but retain a source field, units, validity interval, and confidence where applicable. A later noise experiment may inject measured perception error into simulation observations, but it will be included only after the baseline protocol is complete and its parameters are fixed in advance. The final stage reports variability, failures, ablations, and validity limits alongside successful runs.

5.2 Network and Scenario Preparation

5.2.1 Network Provenance and Validation

A selected, manageable Lahore road area will be exported from OpenStreetMap under its applicable attribution and license conditions [23]. The exact bounding box, extraction date, source file checksum, and conversion command will be stored so that a later change in the public map cannot silently alter an experiment. SUMO's import toolchain will convert the graph, after which NetEdit inspection will verify lane counts, legal turns, junction types, traffic-light programs, pedestrian-only edges, disconnected components, and ambulance-accessible routes. Automated checks will then attempt representative routes, enumerate signal phases, and reject movements that conflict with the intended vehicle classes. The initial planning range is approximately 10–20 signalized intersections, but this is not a confirmed network fact or a requirement to retain a defective import. A smaller validated extract is preferable to a larger inaccurate one because route continuity and lawful signal control are prerequisites for every later result. If the chosen map cannot be stabilized within the schedule, a documented synthetic multi-intersection grid becomes the fallback, and all results will be identified as synthetic rather than generalized to Lahore roads.

5.2.2 Demand and Incidents

Demand files will define light, medium, and heavy conditions together with an uneven-approach case that can expose starvation. Vehicle insertion uses explicit random seeds, and a declared warm-up interval allows traffic to populate the network before measurement begins. Incident scenarios close an edge, reduce lane capacity, or introduce a controlled obstruction at a fixed simulation time. A failed insertion caused by a full lane is retained as an outcome rather than retried until the run appears favorable. The same requested demand and seed are replayed across controllers where SUMO semantics permit. When controller behavior changes route choice or insertion feasibility, the manifest distinguishes the

original demand request from actual departures so that throughput is not interpreted without its denominator. Scenario families are separated into development, tuning, and held-out evaluation sets before final runs. Origin-destination pairs, incident timing, and emergency dispatch time are also matched within each comparison. These controls make differences attributable to the selected policy while preserving congestion effects that legitimately arise from that policy.

5.3 Video Perception Pipeline

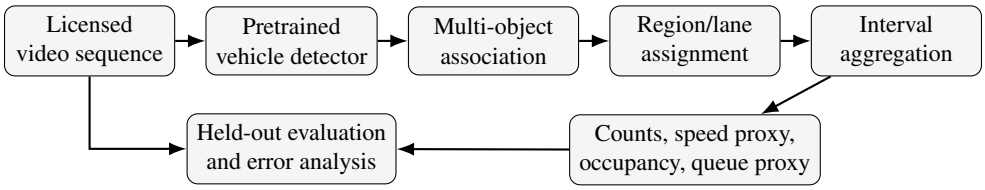


Figure 5.2: Proposed traffic-video perception pipeline

Figure 5.2 presents the complete video-observation chain rather than depicting detection as an isolated result. The detector produces a class, confidence, and image-space box; the tracker associates observations through time; and a bottom-center or calibrated road-contact point maps each valid track to a polygonal lane or approach region. Directed line-crossing or region-entry rules use track history to avoid counting the same vehicle in every frame. Interval aggregation then derives counts, image occupancy, a queue proxy, and a speed proxy with source time and confidence. Held-out annotations feed the error-analysis path and are never used to tune the final evaluation sequence. A speed in physical units requires camera calibration or a defensible known scale. Without that evidence, the system reports pixel displacement or a categorical slow or stationary state rather than kilometres per hour. The figure therefore emphasizes where uncertainty enters the pipeline and why an apparently stable box is not yet a validated traffic measure.

For a lane region l over interval k, a simple occupancy proxy is

$$O_{l,k} = \frac{1}{T_k} \sum_{t \in k} \frac{A\left(\left(\bigcup_{i \in V_{l,t}} B_{i,t}\right) \cap R_l\right)}{A(R_l)}, \tag{5.1}$$

where $B_{i,t}$ is a tracked box, R_l is the image region, and T_k is the number of sampled frames. The union prevents overlapping boxes from being counted twice and keeps the ratio within $[0,1]$. This is image occupancy, not road density. A queue proxy counts stable tracks inside a defined upstream region whose image-motion or calibrated speed is below a threshold for a minimum persistence time.

The detector will be initialized from a documented pretrained checkpoint, consistent with the practical motivation of YOLO-family detection [3]. ByteTrack is the leading tracker candidate because its associ-

ation of lower-confidence boxes may reduce fragmentation under temporary occlusion [5]. DeepSORT remains a comparison when appearance information materially improves association in the selected scenes [4]. Thresholds are tuned only on development sequences and then frozen for held-out evaluation. Selection considers interval count error, track fragmentation when identity is required, queue-proxy error, processing throughput, and failure by scene condition; benchmark reputation alone is insufficient. Equation (5.1) defines image occupancy when calibration is unavailable, using the fraction of a declared region covered by tracked boxes across sampled frames. Because perspective and occlusion affect that fraction, it is reported as a proxy with its region geometry and sampling interval, not renamed as physical road density. This evaluation keeps the claimed output aligned with what the video can actually support.

5.4 Normal Traffic Controllers

5.4.1 Fixed-Time Controller

The fixed controller will use a declared cycle and phase sequence appropriate to each selected junction. Green, yellow, all-red, cycle, and offset values will be versioned with the scenario, and the same movement definitions and clearance rules will constrain every competing controller. The baseline will not be intentionally starved or assigned unrealistic timing to make adaptation appear superior. Initial green splits will be selected from the declared demand and checked against approximate approach service so that at least the light scenario is feasible and heavier scenarios create interpretable congestion rather than immediate gridlock. Its deterministic schedule makes it useful for validating demand generation, metric collection, ambulance route ablations, and replay before learning is introduced. If a run violates phase invariants or produces implausible counts under fixed control, adaptive and reinforcement-learning comparisons are postponed until the cause is resolved. The baseline therefore serves as both a comparator and a diagnostic reference, not as a weak opponent constructed for a favorable result.

5.4.2 Rule-Based Adaptive Controller

The initial transparent controller evaluates only legal candidate phases. A pressure-inspired score for phase p is

$$S_p(t) = \sum_{(l,m)\in p} (q_l(t) - \alpha q_m(t)) + \beta w_l(t), \tag{5.2}$$

where q_l and q_m are upstream and downstream queues for a served movement, w_l is the maximum accumulated wait among its approaches, and α, β are documented configuration values. This follows the motivation of max-pressure control [8] while retaining a direct anti-starvation term.

Equation 5.2 assigns a transparent score to each legal candidate phase using upstream demand, down-

stream congestion, and the longest accumulated approach wait. The downstream term discourages serving a movement whose receiving link lacks space, while the wait term provides a direct safeguard against repeatedly postponing a low-volume approach. Coefficients are configuration values established on development scenarios and retained with the run manifest. At each decision instant, the current green is held until minimum green has elapsed. It may then be extended when it remains the preferred phase and maximum green has not been reached. A phase whose maximum-red limit is approached receives explicit promotion, and every switch inserts the same yellow and all-red sequence used by other controllers. The controller returns a requested phase and its contributing values; it does not actuate SUMO directly. This explanation record permits a reviewer to distinguish an algorithmic request from a safety-filter transformation and to diagnose whether observed performance came from pressure, fairness promotion, or repeated intervention.

5.4.3 Proposed RL Formulation

The initial reinforcement-learning design is a parameter-sharing, discrete-action Deep Q-Network for the selected small network, introduced only after the fixed and rule-based baselines are stable. Each controlled intersection applies the same policy to a fixed-length local observation and requests either one legal phase or a bounded extension. Parameter sharing limits the number of independently trained models and permits consistent logging, while intersection identifiers and movement masks prevent an invalid phase from entering the available action set. The safety layer retains authority over minimum green, yellow and all-red clearance, and maximum-red constraints. Model architecture, observation normalization, replay settings, exploration schedule, optimizer, training budget, and checkpoint-selection rule will be frozen before held-out evaluation. This choice is an initial experimental design rather than a claim that Deep Q-learning is the best controller. If training is unstable, violates fairness limits, or produces no consistent held-out benefit, the result will be retained and the pressure-inspired controller will remain the operational fallback.

The proposed state for each decision interval includes normalized incoming and outgoing queues, approach waiting time, occupancy, current phase, elapsed phase time, and valid-action masks. Emergency status is excluded from this normal-control policy because the corridor is evaluated as a separate supervisory intervention. All queue and waiting inputs use common clipping and normalization ranges fixed during tuning. An action requests a legal phase or a bounded extension. The safety layer remains authoritative. A candidate reward is

$$r_t = -\lambda_p P_t - \lambda_w W_t - \lambda_s X_t - \lambda_c C_t, \tag{5.3}$$

where $P_t = \sum_m |q_m^{\text{in}}(t) - q_m^{\text{out}}(t)|$ is the nonnegative magnitude of movement pressure, W_t is normalized waiting, X_t is a starvation violation, and C_t is an unnecessary-switch penalty. Coefficients and normalization will be fixed on training and tuning scenarios. Emergency performance is evaluated separately so that the normal-controller comparison is not confounded by rare mission rewards.

Equation 5.3 combines network pressure, normalized waiting, starvation violations, and unnecessary switching, with every component logged separately so that one favorable aggregate cannot conceal a harmful term. Coefficients and normalization are chosen on training and tuning scenarios, then frozen. Training records episode seed, demand profile, exploration schedule, optimizer settings, network checksum, reward components, software version, and candidate checkpoints. Checkpoint selection follows a declared tuning criterion rather than the best result on the final scenarios. Evaluation uses deterministic policy settings where applicable and the same safety filter, demand manifests, warm-up, and measurements as the baselines. Training return is a learning diagnostic, not an evaluation result, because it depends on reward scaling and does not directly establish travel time, fairness, or safety. Emergency priority remains a separate supervisory experiment so that rare mission rewards do not confound the first normal-controller comparison. This separation also enables an honest result when a normal controller improves average delay but produces unacceptable maximum-red behavior.

5.5 Dynamic Graph Routing

The routing graph represents legal vehicle movements rather than only geometric road adjacency. Nodes may be junctions or lane and edge transition points, while edges retain length, allowed vehicle class, free-flow time, current travel-time estimate, queue-discharge estimate, residual signal delay, closure state, and observation time. The exact representation must reproduce SUMO route validity, including turn restrictions and internal connections, so that an algorithm cannot return a visually plausible but undrivable path. Every cost field states its unit and source, and stale observations are rejected or replaced by a declared fallback estimate. Equation 5.4 expresses the primary cost in seconds. Link travel time, signal delay, and queue delay are separated only when their estimators do not overlap; if a measured link time already contains queuing, another queue penalty would double-count congestion. A hard closure receives infinite cost, while temporary slowdowns retain finite nonnegative estimates. The complete cost snapshot is versioned with any accepted route so that a later evaluator can reconstruct why it changed.

The primary edge cost is expressed in seconds:

$$c_e(t) = \hat{T}_e^{\text{run}}(t) + \hat{D}_e^{\text{signal}}(t) + \hat{D}_e^{\text{queue}}(t) + M_e(t), \tag{5.4}$$

where $\hat{T}_e^{\text{run}}$ excludes queue and signal delay, and $M_e(t) = 0$ for an open edge and infinity for a hard closure. When a measured total link travel time is available, it replaces the first three terms rather than being added to them. Every cost snapshot records the estimator type, source, and freshness.

Dijkstra's algorithm provides the reference minimum-cost path under nonnegative weights and requires no heuristic assumption [16]. It therefore supplies a clear correctness baseline for route validity and cost even when the selected network is small enough that search speed is not a practical limitation. A* is permitted when its heuristic $h(n)$ is a lower bound formed from straight-line distance divided by the maximum allowed free-flow speed [1]. Under nonnegative signal and queue delays, this estimate remains no greater than the true remaining time and can guide the search without losing the optimality guarantee. The implementation will compare the returned cost with Dijkstra on development graphs and will revert to Dijkstra if admissibility or consistent units cannot be demonstrated. Neither algorithm creates dynamic routing by itself: the cost snapshot, refresh interval, closure handling, current-position mapping, and stability rule determine when a path changes. Algorithm 1 consequently wraps shortest-path search with validation, no-route handling, hysteresis, route versioning, and corridor cancellation.

Data: Current ambulance edge s, destination g, current route R, cost snapshot C_t

Result: Retained or revised valid remaining route

Reject stale or negative costs; apply closures to C_t;

$R' \leftarrow \text{ShortestPath}(s, g, C_t)$;

if R' *does not exist* **then**

    return NO_ROUTE and cancel future corridor requests;

end

Compute remaining costs $J(R)$ and $J(R')$ under the same snapshot;

Validate route continuity from the current position;

if *current route is infeasible* **then**

    mark R' for acceptance;

else

    mark R' only if $J(R) - J(R') > \delta$ for persistence window τ and cooldown has elapsed;

end

if R' *is not marked* **then**

    return the current route unchanged;

end

Atomically increment route version, cancel obsolete priority, and publish R';

Algorithm 1: Congestion-aware route selection and stable rerouting

Algorithm 1 defines route selection and revision as one auditable transaction. It first rejects stale or

negative costs, applies hard closures, and searches from the ambulance's valid current position. Dijkstra provides the reference optimum for nonnegative costs, while A* may be used only with the stated admissible free-flow lower bound. A missing path produces a recorded NO_ROUTE outcome and cancellation of future corridor requests rather than a partial route. When both current and candidate paths are feasible, their remaining costs are evaluated under the same snapshot. The improvement threshold δ, persistence window τ, and cooldown are configured on development scenarios before held-out tests, preventing small fluctuations from causing oscillation. An infeasible current path bypasses the benefit threshold, but the replacement must still pass continuity and vehicle-class checks. Acceptance increments the route version and atomically cancels priority that is no longer justified. Computation time, revision cause, and rejected candidates are retained so route stability can be evaluated alongside ambulance travel time.

5.6 Moving Green Corridor

The moving corridor operates on a bounded number of intersections ahead of the simulated ambulance instead of setting an entire route green. Activation considers estimated arrival, standing queue clearance, signal state, and downstream capacity rather than distance alone. Each request identifies the mission, immutable route version, approach movement, predicted arrival, expiry, and priority level. Before serving that movement, the coordinator completes the active phase's minimum service and applies the configured yellow and all-red clearance to conflicting movements. Passage is confirmed from the ambulance's simulated edge and lane transition, with a timeout detecting a missing or stalled vehicle. The intersection then enters recovery and either completes a bounded compensation plan or returns control at a safe phase boundary. A revised route cancels obsolete future requests before new look-ahead activation, preventing two route versions from competing for a signal. These choices follow the transition and recovery emphasis in emergency-preemption research [18] while keeping all timing values as simulation policies rather than certified field parameters.

Data: Priority request, ambulance state, signal state, safety configuration

if *request is stale, route version is obsolete, or movement is invalid* **then**

    reject and log reason;

end

Queue the request and estimate activation time;

At activation, finish minimum green and apply legal clearance;

Serve the ambulance movement within maximum priority duration;

if *passage is confirmed* **then**

    release and enter recovery;

end

else if *mission is cancelled, route changes, or timeout occurs* **then**

    cancel safely and enter recovery/fallback;

end

Restore normal controller ownership at a valid phase boundary and record recovery time;

Algorithm 2: Corridor lifecycle at one intersection

Algorithm 2 makes the intersection lifecycle explicit. A stale, expired, invalid, or obsolete request is rejected before signal ownership can change. Valid requests wait for their activation condition, complete lawful clearance, and serve the ambulance movement for no longer than the configured maximum priority duration. Confirmed passage, mission cancellation, route revision, or timeout all lead through recovery rather than jumping to an arbitrary normal phase. The core scope permits one active ambulance, avoiding an untested arbitration policy for competing emergencies; multiple emergency vehicles remain future work. Ordinary-traffic harm is bounded through maximum priority duration, downstream occupancy checks, and maximum-red monitoring, and the evaluation records safety-filter interventions and recovery time. Exact limits are selected through development-scenario sensitivity analysis and are reported as simulator settings, not public-road safety values. This lifecycle allows emergency benefit and network disruption to be examined together while ensuring that cancellation is as carefully specified as successful passage.

5.7 Integrated Control Loop

Data: Validated run configuration and initialized services

while *run is active and before terminal simulation time* **do**

Advance SUMO by exactly one configured step;

Read subscribed vehicle, lane, signal, and mission state;

Construct a timestamped traffic-state snapshot;

Update route costs and evaluate rerouting at its configured interval;

Update corridor requests and lifecycle states;

if *corridor owns an intersection* **then**

request its safe priority action;

else

request action from the selected normal controller;

end

Validate every requested action through the phase safety filter;

Apply accepted actions through the TraCI adapter only;

Record applied actions, events, and metric samples;

Publish a non-authoritative dashboard snapshot;

end

Finalize run status, artifacts, and checksums;

Algorithm 3: Single-owner simulation and decision loop

Algorithm 3 integrates normal control, route updates, and corridor ownership under one simulation-clock writer. Each iteration advances SUMO once, reads a complete observation set, creates a timestamped snapshot, and then performs routing and corridor updates at their configured intervals. For each intersection, the corridor or the normal controller can request an action, but both pass through the same safety filter and only the TraCI adapter applies the result. Applied actions, metric samples, and mission events are recorded before a dashboard view is published. This ordering prevents a slow client, background video job, or concurrent controller from advancing simulation time or acting on a partially updated state. A snapshot carries its effective step, production time, source, and validity interval so late data is deliberately mapped or rejected. At termination, the manifest and checksums are finalized even for stopped or failed runs. The loop therefore defines both concurrency ownership and the causal order needed to replay an experimental decision.

5.8 Experimental Protocol

5.8.1 Comparison Matrix

Table 5.1 maps each evaluation objective to a controlled comparison and the evidence required to assess it. Perception is compared with held-out annotations, normal control uses matched scenarios and seeds, routing holds the signal policy fixed, and corridor evaluation holds the route logic fixed. This staged design prevents a combined emergency result from being credited to the wrong component. The robustness row then stratifies demand, incidents, seeds, and any approved observation-noise conditions, retaining failures and no-route outcomes. Measures are selected to reveal tradeoffs: average delay is accompanied by queue extremes and fairness, ambulance time is paired with ordinary-traffic impact and recovery, and route benefit is paired with revision count and computation cost. The table is a plan rather than a results summary. No numerical improvement, accuracy, sample count, or significance is asserted until the implementation produces traceable run records under this protocol. Final result tables will preserve these objective and measure definitions.

Table 5.1: Planned experiments and primary evidence

Objective	Controlled comparison	Primary measures
Perception quality	Detector/tracker output vs. held-out sequence annotations	Precision/recall or mAP where suitable; count MAE; identity metric only when required; queue-proxy error; throughput
Normal control	Fixed vs. rule-adaptive vs. selected RL under matched scenarios/seeds	Mean travel and waiting time, queue mean/max, throughput, phase changes, max-red/fairness violations
Dynamic routing	Static/reference route vs. dynamic route, holding signal policy fixed	Ambulance travel time, delay, stops, route revisions, computation time
Moving corridor	Same routing without vs. with corridor	Ambulance time/stops; ordinary traffic delay/queues; recovery time; safety invariant violations
Robustness	Demand, incident, seed, and optional observation-noise strata	Effect distributions and sensitivity; failed and no-route runs retained

5.8.2 Scenario Assignment and Repetition

The initial evaluation protocol uses a 900-second warm-up followed by a 3,600-second measurement interval for every matched simulation run. Each controller and scenario combination receives the same ten demand seeds, origin-destination pairs, incident timing, and ambulance dispatch time. A pilot may justify increasing the number of matched seeds when variability is high, but it cannot reduce the predeclared set or remove an unfavorable run. These values will be revised only before evidence collection if network

validation shows that the warm-up has not reached a stable loading pattern; the reason and replacement values will then be recorded in the frozen protocol. Results report paired controller differences, standard deviation, and confidence intervals where their assumptions are satisfied. Reinforcement-learning training and checkpoint selection use only training and tuning scenarios, while unseen demand combinations and a disjoint seed set remain reserved for final evaluation. Failures, timeouts, infeasible routes, and safety interventions stay in the run inventory and are reported separately when aggregation would obscure their meaning.

5.8.3 Metric Definitions

Metric definitions are shared across controllers and versioned with the run. For N completed vehicles with travel times T_i, mean travel time is $\bar{T} = N^{-1} \sum_i T_i$. Queue length is sampled at fixed simulation intervals and reported as both time mean and maximum by approach. Throughput counts vehicles crossing declared network boundaries during the measurement interval, while the demand manifest and actual departures prevent a low completion count from being misread. Fairness includes maximum red time and the approach-level delay distribution because an aggregate mean can conceal starvation. Ambulance travel time is the primary emergency measure. Excess delay is the observed travel time minus the free-flow time of the same realized edge sequence, while route benefit is assessed separately by comparing matched routing policies. This distinction prevents a longer selected path from receiving an artificial advantage through an unrelated reference path. Ordinary-traffic impact covers non-emergency vehicles during the mission and the declared recovery interval. Route computation time is measured separately from simulated travel time, and revision count identifies unstable routing. For perception counts y_k and estimates $\hat{y}_k$, mean absolute error is

$$\mathrm{MAE} = \frac{1}{K} \sum_{k=1}^{K} |y_k - \hat{y}_k|. \tag{5.5}$$

5.8.4 Reproducibility Package

Each run will retain the source revision, dependency manifest, SUMO version, network and route checksums, controller and model version, full configuration, seed, logs, metric schema, and terminal reason. Applied decisions are stored separately from controller requests, and route revisions identify the cost snapshot and trigger that produced them. Large external video and pretrained weights may be referenced by a license-aware URI and checksum when redistribution is not permitted. A script should recreate a run from its manifest, and every manual NetEdit change must be committed as a versioned network artifact rather than left in an operator's local state. Derived plots and summary tables retain their source run identifiers and analysis-script revision. Failed and cancelled runs keep their partial logs

and explicit terminal state. This package does not guarantee identical hardware timing, but it makes scenario semantics, causal events, and analytical transformations inspectable enough for another team member to reproduce the experiment or explain a difference.

5.9 Threats to Validity

Construct validity is threatened when a queue proxy, waiting-time implementation, or reward component does not represent the concept named in the report. Definitions and units are therefore versioned, and image-space proxies are not relabeled as physical quantities without calibration. Internal validity can be weakened by unmatched demand, different warm-up, hidden seed changes, or controller-specific failures; paired manifests, one measurement implementation, and retention of failed runs reduce these risks. External validity remains limited by synthetic demand, map inaccuracies, SUMO car-following assumptions, a small selected network, and domain shift between public video and Lahore traffic. Conclusions will be bounded to the tested network, demand families, software versions, and video sources, with no claim about clinical response time or public-road safety. Conclusion validity can be weakened by too few repetitions, test-set tuning, selective failure removal, or reporting only means. Pilot variance, held-out scenarios, paired comparisons, variability, and sensitivity analysis address those concerns. If statistical tests are used, their assumptions and effect sizes will be reported; significance alone will not establish practical value.

5.10 Conclusion

The methodology specifies a staged progression from validated measurements and baselines to integrated emergency control. Network provenance and demand manifests support repeatable scenarios; source-separated video evaluation prevents perception estimates from being confused with simulator truth; and common safety constraints permit a fair controller comparison. Routing uses time-consistent nonnegative costs, validity checks, and hysteresis, while the corridor lifecycle makes request, clearance, active priority, passage, cancellation, and recovery observable. The comparison matrix separates perception, normal control, routing, corridor effect, and robustness so the cause of a result can be explained. Held-out scenarios, repeated seeds, explicit metrics, retained failures, and reproducibility artifacts protect the final evaluation from undocumented tuning. Most importantly, the procedure allows a learned controller, tracker, rerouting policy, or corridor configuration to produce a negative or mixed finding without prompting selective changes to the test protocol. The next chapter maps these methodological responsibilities to components, interfaces, state ownership, and failure handling.

Chapter 6 High-Level and Low-Level Design

This chapter maps the requirements and methodology to a service-oriented prototype design that can be implemented incrementally on the team's available machines. The decomposition protects experimental determinism by assigning the simulation clock and all signal actuation to one adapter. The API, dashboard, perception worker, controllers, routing logic, and experiment store exchange typed messages without competing for that authority. The design also separates a requested decision from the action applied after safety checks, preserves the source and effective time of every traffic snapshot, and binds corridor requests to an immutable route version. These choices support replay and fault diagnosis without requiring a prematurely distributed deployment. Interfaces may initially be implemented as in-process modules, provided that they retain the same ownership boundaries and contracts. The chapter moves from architecture and domain objects to interaction sequences, state transitions, data contracts, synchronization, failure handling, module policies, and testing so each important requirement has a visible design consequence.

6.1 System Overview

The system must support repeatable research runs, an interactive demonstration, and post-run analysis without allowing presentation concerns to alter experimental state. Its main design goals are deterministic ownership of state changes, replaceable controller and perception implementations, explicit time and units, safe corridor transitions, immutable run configuration, and visible failure. Performance is optimized only after these properties are preserved because a faster result is not useful when its source, phase legality, or causal order is uncertain. The design assumes one machine or a small development deployment for Deliverable 2 and FYP-I. Logical separation therefore does not imply a network service for every box in the diagrams. The orchestrator, controllers, routing logic, and repositories may begin as in-process modules behind typed interfaces, while video inference can move to a bounded worker when profiling shows it would block the control loop. This approach keeps deployment manageable but preserves a path to isolate resource-heavy components without changing experimental semantics.

6.1.1 Operator Interface and Commands

The operator uses a React dashboard for configuration, monitoring, emergency, and analysis tasks. Commands pass through FastAPI validation before reaching the experiment service or orchestrator. The interface cannot step SUMO, apply a phase, or alter an active run's immutable configuration. Responses include source, simulation step, freshness, and status so the dashboard can distinguish current evidence from a stale or unavailable view. Configuration commands resolve network, controller, seed, duration,

incident, mission, and metric identifiers before a run begins. Monitoring queries expose requested and applied actions separately, while emergency commands return the route version and corridor state affected by the request. The evaluator can inspect completed and failed runs without receiving mutation privileges. A disconnected browser may reconnect from the current snapshot and event cursor, but it cannot replay an old command automatically. These rules make the graphical interface useful for demonstration and review without creating a second, poorly controlled path into the traffic simulation.

6.1.2 Traffic Observation and Simulation

SUMO provides authoritative traffic state for controlled experiments, while a bounded perception worker supports recorded-video observation. The traffic-state service normalizes either source without presenting video proxies as simulator measurements. Only the SUMO/TraCI adapter owns simulation stepping, subscriptions, and applied signal changes. Perception reports model version, input provenance, confidence, and freshness, preserving a clear record of what information was available to each decision. A simulator snapshot can include exact vehicle, lane, signal, and route values available through TraCI. A video snapshot contains only quantities justified by the selected detector, tracker, road regions, and calibration; missing physical values remain unavailable instead of being replaced by zero. Both snapshots identify the time at which values became effective and the period for which they remain valid. The control loop uses one declared source mode per experiment, while offline perception jobs can be assessed independently against labelled footage. This boundary prevents the report from treating visual detection output as equivalent to controlled simulator ground truth.

6.1.3 Adaptive Control and Emergency Routing

Normal traffic control and emergency priority share the safety filter but remain separate decision concerns. Fixed-time, pressure-inspired, and learned controllers return requested actions through a common interface. Routing creates immutable, time-costed ambulance paths, and the corridor coordinator translates the current route into bounded priority requests. Neither module actuates a signal directly; every request must satisfy legal phase, clearance, ownership, expiry, and recovery rules. The common controller interface supplies the same snapshot, legal phases, decision interval, and explanation fields to each normal strategy so comparison is not distorted by different privileges. Emergency operation is supervisory: it temporarily owns selected intersections through an explicit state machine and returns them to a valid normal boundary after passage, cancellation, or timeout. Rerouting creates a new route version and cancels obsolete future requests before activating the revised look-ahead set. Separating route choice, corridor priority, normal control, and safety enforcement permits useful ablation experiments and makes a rejected decision traceable to its responsible component.

6.1.4 Experiment and Evaluator Interaction

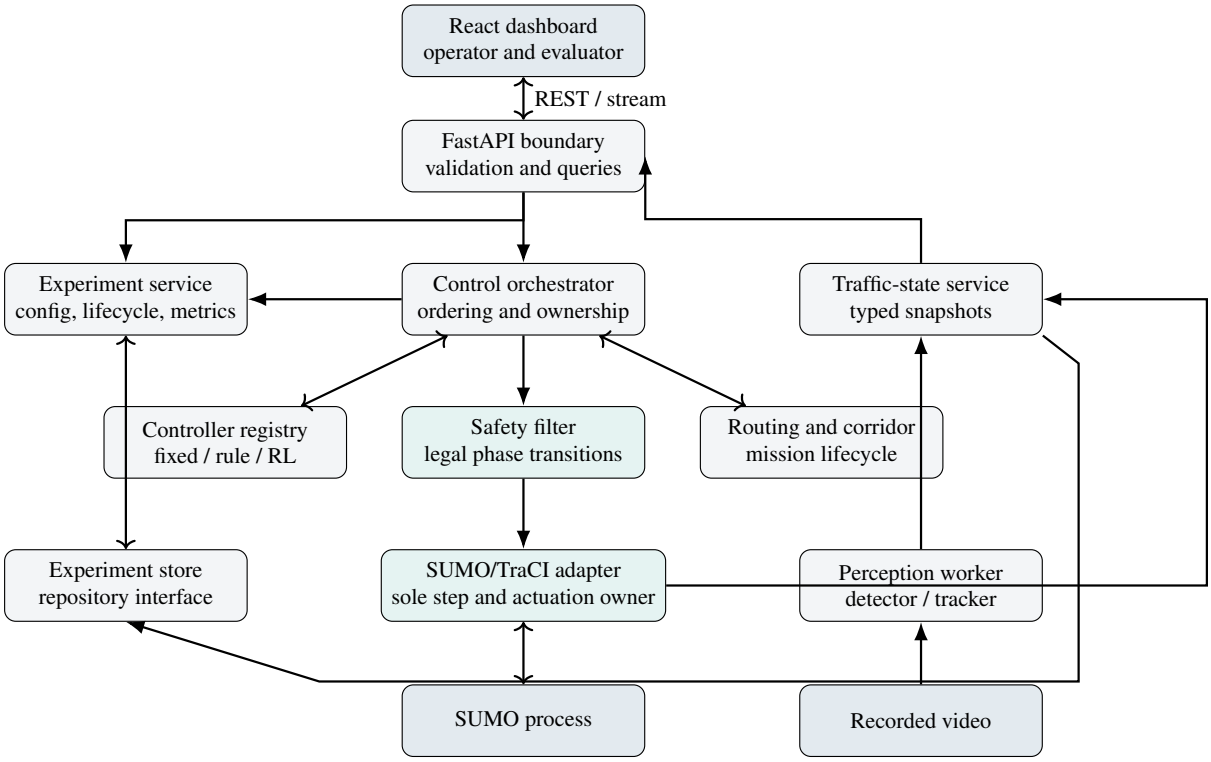


Figure 6.1: Proposed high-level architecture

Figure 6.1 shows the command and evidence paths around the single-owner simulation core. The React dashboard sends configuration or mission commands through FastAPI and receives non-authoritative views; it never calls TraCI. The orchestrator follows Algorithm 3, requests decisions from the selected normal controller or corridor coordinator, and sends validated actions to the adapter. Only that adapter advances SUMO and applies signal changes. The traffic-state service constructs source-specific snapshots from simulator observations or perception messages while preserving effective time, units, confidence, and validity. Routing maintains mission and route versions, and the corridor coordinator owns intersection-level priority lifecycles. The experiment service validates run configuration, records events and metrics, and persists them through a repository interface whose proposed implementation is PostgreSQL. Directional arrows in the figure distinguish state flow from command ownership, making clear that perception can inform a snapshot without acquiring authority over simulation time. This separation also keeps dashboard latency outside the control path.

Table 6.1 assigns each component the data it consumes and the state or evidence it is permitted to produce. This allocation prevents shared mutable state from becoming an undocumented integration mechanism. The dashboard owns only presentation state, controllers own only proposed actions and explanations, and the SUMO adapter owns the external process, subscriptions, clock, and applied actuation. The orchestrator orders these modules but does not replace their domain logic. Routing and

corridor responsibilities are listed separately because a shortest path does not imply authority to change a signal, and a corridor request must retain the route version that justified it. The traffic-state service can normalize observations from two source modes but may not silently combine missing video measures with exact simulator values. Finally, the experiment store receives validated records and artifacts rather than arbitrary objects from every module. These boundaries give each requirement a testable owner and permit an implementation to remain modular even when several components share one process.

Table 6.1: Component responsibility allocation

Component	Consumes	Produces / owns
React dashboard	Query responses, event snapshots, user input	Presentation state; no control authority
FastAPI boundary	HTTP/stream commands, identity-free local operator session	Validation, command/query boundary, error mapping
Experiment service	Valid configuration, metric/event batches	Run lifecycle, immutable manifest, comparison eligibility
Control orchestrator	Run config and ordered state snapshot	Decision schedule, module ordering, requested actions
SUMO/TraCI adapter	Validated step and action commands	Simulation process, clock, subscriptions, applied actions
Traffic-state service	SUMO observations or perception messages	Versioned source-specific snapshots and freshness
Perception worker	Video and model configuration	Detections, tracks, aggregates, job health
Controller registry	State, legal phases, timing constraints	Requested normal-control action and explanation
Routing service	Graph, mission, cost snapshot	Route versions, no-route reason, reroute decision
Corridor coordinator	Route version, ambulance state, signal state	Priority request/state, release and recovery requests
Experiment store	Validated records and artifacts	Durable configuration, events, metrics, checksums

6.2 Design Considerations

6.2.1 Assumptions and Dependencies

The design assumes a validated SUMO network, legal traffic-light programs, reproducible demand files, and approved recorded video when perception experiments are performed. It depends on TraCI for simulator control and on versioned model artifacts and repositories for repeatable evidence. A single development machine is sufficient for initial integration, although detector training may require separate GPU capacity. Missing maps, checkpoints, or video sources therefore produce explicit fallbacks or blocked experiments rather than hidden substitutions. Network preparation must resolve disconnected edges, prohibited movements, signal programs, and ambulance-compatible routes before a scenario is accepted. Demand generation exposes its seed and distribution parameters so repeated runs can be paired. Video processing assumes that clip provenance, model checksum, road regions, and any calibration are available. The repository must retain configuration and causal events even when large artifacts remain external files. These assumptions are verified at configuration time where possible; unmet conditions become validation errors, declared limitations, or a documented synthetic-network fallback rather than undocumented changes to the experiment.

6.2.2 General Constraints

The prototype cannot assume city-wide live sensors, traffic-authority deployment, or emergency-service integration. Evaluation is simulation based, with recorded video used only for perception validation. The core scope permits one active emergency mission, every signal transition must follow the validated program, and a run's configuration becomes immutable at start. Failed runs remain visible, while no dashboard action can bypass backend validation. These constraints prevent a polished demonstration from being confused with operational deployment. The selected Lahore network must remain small enough for repeated seeded experiments and readable inspection of imported connections. The system does not claim certified preemption, field latency, legal authority, or direct control of real signals. Learned controllers operate within the same legal action space and decision schedule as baselines, and training-only tuning cannot use held-out evaluation scenarios. Resource limits may require offline detector inference, compact networks, or a documented synthetic grid. Such reductions are acceptable only when the report discloses them and conclusions remain bounded to the tested network, demand, footage, models, and computing environment.

6.2.3 Goals and Guidelines

The design prioritizes traceability, deterministic ordering, safe actuation, replaceable research components, and honest reporting of unavailable evidence. Each mutable resource has one owner; messages carry version and effective time; requested and applied actions remain distinct; and fallback behavior is bounded and labelled. Interfaces are small enough for contract testing, and diagrams describe logical responsibilities rather than claiming every component is already deployed as a separate service. Naming, units, source modes, and terminal states remain consistent across the API, database, logs, interface, and report. A component rejects an unknown schema, expired snapshot, illegal phase, disconnected route, or incompatible comparison rather than trying to infer intent. Experiments retain failures, exclusions, and fallback intervals so favorable summaries cannot hide them. Performance optimization follows profiling and must not bypass validation or remove evidence. A change is accepted only when its owner, inputs, outputs, time semantics, failure behavior, and effect on reproducibility can be stated and tested.

6.2.4 Development Methods

Table 6.2 records the main alternatives considered and the consequence of each selected approach. The choices favor traceability and feasible integration over maximum distribution or algorithmic novelty. SUMO remains the simulator because it aligns with the approved proposal and provides detailed route and signal inspection, although CityFlow could offer greater training throughput for a different scope. A common controller interface and safety filter make fixed, rule-based, and learned policies comparable, while supervisory emergency control permits a separate corridor ablation. Dijkstra remains the routing reference and A* is conditional on an admissible time lower bound. Structured persistence is proposed because flat files make cross-run relationships difficult to enforce, but the repository boundary keeps the engine replaceable. The decision record is not evidence that every selected technology has been completed; it explains why the design adopts these constraints and what cost each creates, including additional message schemas, job states, and performance profiling.

Table 6.2: Architectural decision record

Decision	**Selected approach and consequence**	**Alternative considered**
Simulation ownership	One orchestrator and adapter control stepping and signals, giving a deterministic replay and concurrency model.	Multiple services calling TraCI risk non-deterministic order.

Decision	Selected approach and consequence	Alternative considered
Simulator	SUMO aligns with the proposal and supplies detailed road and signal tooling; performance is profiled later.	CityFlow offers training throughput but changes the approved environment.
API and interface	FastAPI and React provide a typed boundary and independent interface iteration.	A server-rendered monolith is simpler to deploy but weakens service contracts.
Controller integration	A common request interface and safety filter make controllers comparable and preserve applied-action evidence.	Direct TraCI access couples research policies to actuation.
Emergency control	A supervisory corridor state machine supports a clear ablation and one safety owner.	Folding emergency reward into normal RL confounds attribution.
Routing	Dijkstra is the reference; A* is permitted only with an admissible time lower bound.	Distance-only routing does not satisfy the congestion-aware evaluation objective.
Persistence	A structured experiment schema remains portable behind a repository interface.	Flat files are quick but weak for cross-run integrity.
Perception execution	A bounded background worker prevents inference from blocking control while exposing job state.	In-request inference couples API latency to model runtime.

6.3 System Architecture

The Manzil architecture follows the component boundaries introduced in Section 6.1.4 and refines them into objects that can be implemented and tested independently. The dashboard and API form the presentation boundary, while the orchestrator, traffic-state service, controller registry, routing service, corridor coordinator, safety filter, simulation adapter, and experiment store form the application core. This partition keeps visualization outside the authoritative control path and assigns every mutable resource to one owner. Communication may initially use in-process calls, but each interaction retains a typed request

or event contract so that later separation does not change experimental meaning. The architecture also distinguishes a proposed action from an applied action, a traffic observation from its source, and a route from the version that authorized corridor activity. These distinctions make replay and fault diagnosis possible because each state change can be connected to its input snapshot, policy, safety decision, and effective simulation step. The following domain diagram records these relationships without committing the implementation to one framework or database library.

6.3.1 High-Level Partitioning

The presentation, application, research, infrastructure, and evidence responsibilities form the principal partitions. React and FastAPI form the presentation boundary; the orchestrator and traffic-state service coordinate the application flow; and controllers, perception, routing, and corridor logic contain replaceable research behavior. The SUMO adapter and repositories isolate external infrastructure, while the experiment service connects every partition to durable evidence. Figure 6.1 shows their communication and Table 6.1 states the data each partition may own. Partition boundaries are logical rather than mandatory network processes. During initial integration, several application modules may run inside one Python process, provided that they exchange the same validated objects and respect ownership. The perception worker may be isolated when inference affects timing, and the dashboard remains a separate client because its refresh rate must not control simulation progress. This partitioning provides a manageable deployment for the team while retaining clear seams for profiling, replacement, testing, and later distribution.

6.3.2 Component Collaboration

Collaboration follows typed requests and events instead of shared mutable objects. At each step the adapter supplies state, the traffic-state service creates a snapshot, the selected controller proposes an action, the safety filter validates it, and the adapter applies the result. Routing publishes immutable route versions to the corridor coordinator. The experiment service records configuration, actions, mission changes, and metrics. Section 6.6 makes this ordering explicit for normal control, dispatch, and rerouting. Every request carries identifiers that link it to a run, simulation step, controller or mission, and schema version. The receiving component either produces a validated response or a structured rejection; it cannot silently repair an incompatible message. Events are delivered to persistence in causal order, while dashboard streams may omit intermediate views without changing the authoritative record. These collaboration rules make integration failures observable and allow stubs to replace unfinished modules during contract testing without bypassing the same validation applied to completed components.

6.3.3 Rationale for Decomposition

The decomposition isolates research algorithms, external tools, and presentation concerns. A controller or detector can change without receiving direct TraCI authority, while a storage engine or interface library can change without altering the meaning of a snapshot or applied action. The split also permits bounded perception execution and deterministic simulation. These benefits justify explicit contracts because reproducibility and fault diagnosis are central evaluation requirements. A monolithic script could produce an early visual demonstration, but it would make phase ownership, failure containment, and comparative evaluation difficult to verify. Conversely, deploying every box as an independent service would add network failure and configuration overhead before profiling shows a need. The selected middle position preserves conceptual services behind interfaces while permitting in-process execution. It supports incremental development: fixed control and simulation can operate before perception or learning is ready, and later modules can be added without rewriting the control loop or database meaning.

6.3.4 System Architecture Diagram

Figure 6.1 presents the proposed architecture. Its central feature is the single actuation path from orchestrator through safety filter and SUMO adapter. State and evidence travel outward to the traffic-state and experiment services, while the dashboard remains a non-authoritative observer and command source. This direction prevents interface refreshes, detector delays, or database queries from becoming undocumented signal commands. The figure also separates routing from corridor ownership: a route is computed from a versioned cost snapshot, but only the corridor coordinator may request temporary priority along that route. Perception contributes observations through the traffic-state service and cannot call a controller or adapter directly. The experiment service receives lifecycle and evidence records but does not determine a phase. Readers should therefore interpret arrows as permitted command or evidence paths, not merely as data availability. Missing arrows are intentional constraints that preserve determinism and traceability.

6.3.5 Subsystem Architecture

Each subsystem exposes a narrow testable interface. Controllers return proposed actions, routing returns versioned paths, the corridor coordinator returns route-bound priority requests, and the traffic-state service returns source-specific snapshots. Infrastructure adapters translate these domain objects to TraCI, HTTP, streams, and persistent records. The domain model in Section 6.5 describes the stable objects shared across those interfaces. The simulation subsystem owns process launch, stepping, subscriptions, and applied commands. The experiment subsystem owns configuration validation, lifecycle status, man-

ifests, metrics, and comparison eligibility. The perception subsystem owns jobs, model provenance, detections, tracks, and derived region observations. The emergency subsystem owns missions, route versions, priority requests, passage, and recovery. These responsibilities prevent two modules from writing the same state. Each subsystem can be exercised with fixtures representing adjacent contracts, allowing tests to distinguish local logic errors from integration or external-tool failures.

6.4 Architectural Strategies

6.4.1 Choice of Technologies

Python, SUMO, TraCI, FastAPI, React, and structured persistence match the approved proposal and divide research computation from presentation. Python supports controllers, routing, perception, and analysis; SUMO supplies reproducible traffic dynamics; FastAPI defines validated contracts; and React supports independent dashboard views. Versions are written to run manifests. Final detector, tracker, reinforcement-learning library, and database-engine choices remain configuration decisions until implementation evidence confirms them. Selection also reflects team feasibility: these technologies run locally, expose documented interfaces, and support automated testing without requiring proprietary traffic infrastructure. SUMO is retained over a throughput-oriented alternative because its network, vehicle, signal, and TraCI tooling align with the approved scope. FastAPI and React add a service boundary but allow the research core and interface to evolve independently. A technology is not accepted solely because it is popular; its version, configuration, licence, input assumptions, and effect on timing or metrics must be recorded for the experiment that uses it.

6.4.2 Reuse of Existing Software Components

Manzil reuses SUMO traffic models, TraCI primitives, OpenStreetMap conversion tools, computer-vision libraries, and standard graph algorithms. Adapters bound this reuse so an external package cannot define project semantics implicitly. Imported networks are checked for connectivity and signal legality, model checkpoints retain provenance, and route costs state their units and observation time. Manzil contributes experiment, safety, corridor, audit, and comparison logic that combines these components consistently. Reuse reduces implementation risk but does not transfer responsibility for validation to a library. The network importer must still confirm turn connections and traffic-light programs; the perception pipeline must still evaluate selected checkpoints and tracking rules on the intended footage; and the routing service must still build a legal vehicle-movement graph. Third-party defaults are captured in manifests when they affect results. External components are accessed through project-owned interfaces so a library upgrade or replacement produces an explicit compatibility change instead of silently altering

stored evidence or control behavior.

6.4.3 Future Extension and Enhancement

Versioned contracts and repository interfaces allow later support for additional corridors, several emergency vehicles, new observation sources, distributed training, or a different simulator. These are extension points rather than current capabilities. Any extension must preserve single ownership, causal event ordering, legal phase transitions, and explicit comparison manifests so increased scale does not weaken the evidence model. Multiple emergency vehicles would require a conflict-resolution policy rather than simply creating more corridor requests. A live camera source would require privacy review, calibration, clock synchronization, and monitored failure behavior. Distribution would require authenticated service boundaries and a stronger event-delivery model, while a simulator replacement would require an adapter capable of reproducing the same snapshot and applied-action semantics. Keeping these possibilities outside the current claim avoids implying unfinished functionality, yet documenting their architectural seams prevents short-term choices from closing a reasonable development path.

6.4.4 Version Control

Git maintains source code, schemas, scenarios, and report figures. Each retained experiment records its commit identifier, dependency lock, model checksum, network and demand versions, controller configuration, and metric schema. Demonstration and result releases are tagged. Generated outputs do not replace source configuration, and secrets or machine-specific paths are excluded from portable manifests. Feature work is reviewed through branches or equivalent isolated changes before it is incorporated into an evidence-producing release. Schema and contract changes require coordinated fixtures and migration notes; a commit that changes the meaning of a metric also increments its definition version. Large model, video, and run artifacts are referenced by checksums rather than committed indiscriminately. This policy links every reported result to an inspectable software state while keeping the repository manageable. It also allows an evaluator to distinguish an exact reproduction from a later rerun made after code, data, or configuration changed.

6.4.5 User Interface Paradigm

The interface uses task-oriented screens and a unidirectional command-and-view model. Operators configure runs, monitor network and mission state, inspect provenance, and review evidence without directly manipulating the simulator. Validation errors are tied to fields; colors have textual labels; units and freshness remain visible; and measured results remain distinct from schematic values. These rules implement Section 4.8 and permit browser reconnection without transferring control ownership. Screen identifiers

and navigation reflect the user's workflow from configuration through monitoring to comparison, while specialist details remain available without crowding the overview. Emergency state is displayed with route version, lifecycle state, and warnings instead of a single ambiguous priority indicator. Analytics disclose sample count, variability, failed runs, metric definition, and configuration differences near each comparison. The interface therefore supports operational awareness and academic review, but it never presents unimplemented fields, schematic charts, or stale streams as measured evidence.

6.4.6 External Database and Data Storage Management

Structured configuration, events, routes, and metrics follow Section 4.9; videos, models, and logs remain checksummed external artifacts. A repository interface permits file-backed integration and later PostgreSQL use without changing domain meaning. Schema migrations, referential constraints, immutable completed-run configuration, and versioned metrics protect comparison integrity. Bounded buffering may tolerate a temporary write failure, but processing stops before evidence loss becomes unbounded. The database records identifiers and provenance for external artifacts so a run does not depend on an unexplained local filename. Observation ownership is exclusive, route versions increase monotonically, and priority events retain the route that justified them. Applied control records are authoritative for evaluation because a safety intervention may differ from the controller request. Backup and export procedures include manifests and schema versions, while credentials and licensed content remain outside research exports. These measures keep storage decisions aligned with auditability rather than treating persistence as a passive logging concern.

6.4.7 Concurrency and Synchronization

The orchestrator is the only simulation-step owner, and TraCI access is serialized through the adapter. Controllers receive immutable snapshots and return requests instead of mutating shared state. Perception uses a bounded queue, dashboard streams may skip views without delaying control, and route publication is atomic with cancellation of obsolete corridor work. Shutdown rejects new commands, terminates or detaches SUMO safely, flushes records, and assigns an explicit terminal state. Simulation time, monotonic event sequence, and wall-clock time are stored separately because inference and persistence delays must not be mistaken for traffic causality. A late observation is marked stale or omitted according to policy instead of blocking the loop indefinitely. The active route version changes together with invalidation of its old requests, preventing two paths from owning the same corridor. Database batches may improve throughput, but applied actions and mission transitions must first enter a durable bounded queue. These rules define synchronization even when all components run on one machine.

6.5 Domain Model/Class Diagram

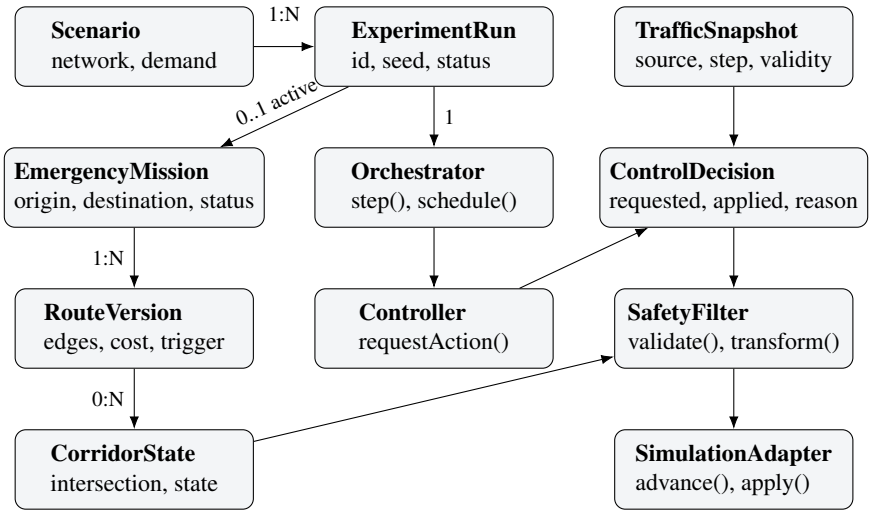


Figure 6.2: Proposed domain model/class diagram

Figure 6.2 presents the core domain objects and their relationships independently of a particular web or database framework. A Scenario can produce many ExperimentRun records, while each run has one orchestrator that schedules controllers and the sole simulation adapter. The model deliberately separates a ControlDecision's requested phase from its applied phase. A controller can request an early or otherwise invalid switch, but the SafetyFilter validates or transforms that request before the adapter can reach SUMO. TrafficSnapshot carries source, simulation step, and validity so a decision can be traced to the observation it used. A run has at most one active EmergencyMission in the core scope. That mission owns immutable RouteVersion objects, and each version can justify several CorridorState records. Connecting the corridor to the same SafetyFilter makes clear that normal and emergency actions share legal-transition enforcement. Exact method signatures may change during implementation, but the cardinalities, single-writer path, route versioning, and requested-versus-applied distinction must remain intact.

6.6 Sequence Diagrams

6.6.1 Normal Control Update

Figure 6.3 shows the causal order for one normal-control decision. The orchestrator first asks the adapter to advance one simulation step, after which raw subscriptions are normalized into a complete snapshot. Only then does the selected controller receive state and return a phase request. The orchestrator validates that request against phase membership, elapsed minimum green, clearance, and ownership before the adapter applies it. The resulting event passes through the experiment service for schema validation

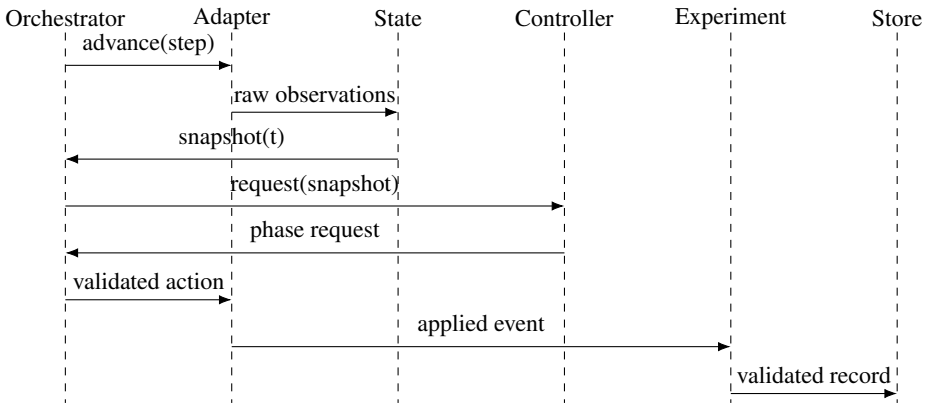


Figure 6.3: Sequence for a normal signal-control update

and run association before the repository stores it. A controller therefore never observes a partial update, cannot actuate SUMO as a side effect, and cannot write an unvalidated evidence record. When a controller decision interval spans several simulation steps, the last applied action remains in force while state and metrics continue to be sampled at their declared intervals. The stored event includes requested and applied action, effective step, snapshot step, and intervention reason, allowing replay to distinguish a policy choice from a safety transformation.

6.6.2 Ambulance Dispatch and Corridor Activation

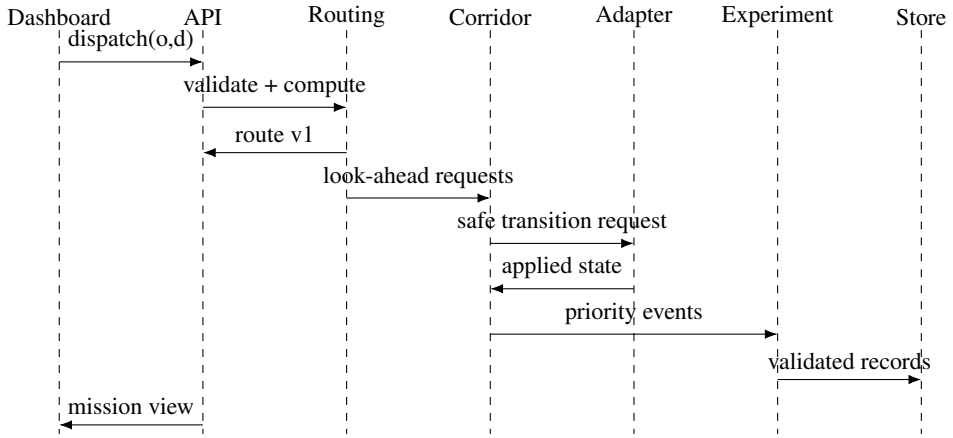


Figure 6.4: Sequence for ambulance dispatch and corridor activation

Figure 6.4 begins when the operator submits an origin and destination through the dashboard. FastAPI validates the command and asks the routing service to resolve accessible edges and compute the first immutable route version. A successful route is returned for the mission view and supplied to the corridor coordinator, which creates bounded look-ahead requests rather than activating the whole path. Each due request passes through safe transition before the adapter applies priority. The corridor sends mission and priority events to the experiment service, which validates their identifiers and route versions before persistence. Invalid endpoints or a disconnected graph stop the sequence before an ambulance or

partial corridor is activated. The dashboard receives a derived mission view after authoritative state has changed, so disconnecting the client cannot cancel or delay the mission. The sequence separates route computation, signal ownership, evidence validation, and presentation while retaining the identifiers required to trace an active priority state back to the command and route version.

6.6.3 Rerouting and Recovery

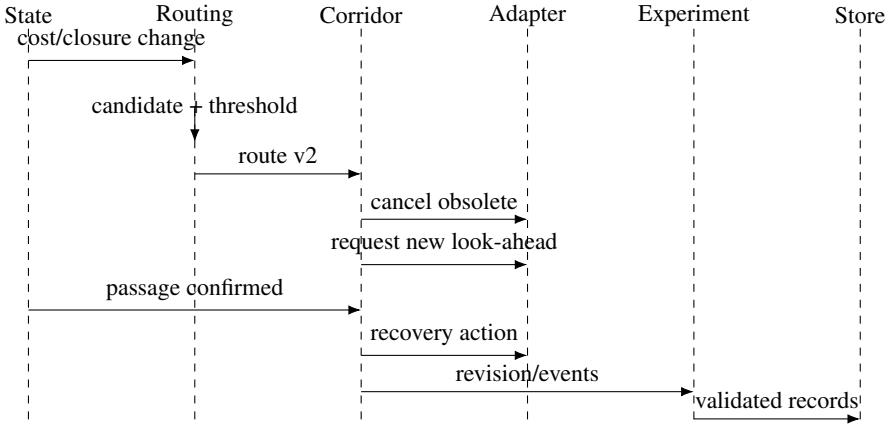


Figure 6.5: Sequence for route revision, obsolete-priority cancellation, and recovery

Figure 6.5 shows how a closure or sustained cost change is handled after dispatch. The routing service evaluates a candidate from the ambulance's valid current position and applies the configured feasibility, benefit, persistence, and cooldown rules. If accepted, a new immutable route version is published to the corridor coordinator. Requests associated with the old remaining path are cancelled before the new look-ahead set is activated, preventing simultaneous priority for incompatible route versions. Passage confirmation still originates from traffic state and moves the relevant intersection into recovery, while route revisions, cancellations, requests, and recovery events are stored in causal order. When no feasible replacement exists, the mission becomes blocked and upcoming priority is cancelled rather than preserving a corridor that no longer reaches the destination. This interaction makes rerouting an auditable state transition, not merely a line redrawn on the dashboard. Rejected candidates also remain available for diagnosis.

6.7 Policies and Tactics

6.7.1 Corridor State and Safety Policy

Figure 6.6 defines the lifecycle owned by each affected intersection. IDLE accepts a valid route-versioned request, but REQUESTED does not imply signal ownership. A request that expires or becomes invalid at this stage returns directly to IDLE because no signal state has changed and recovery

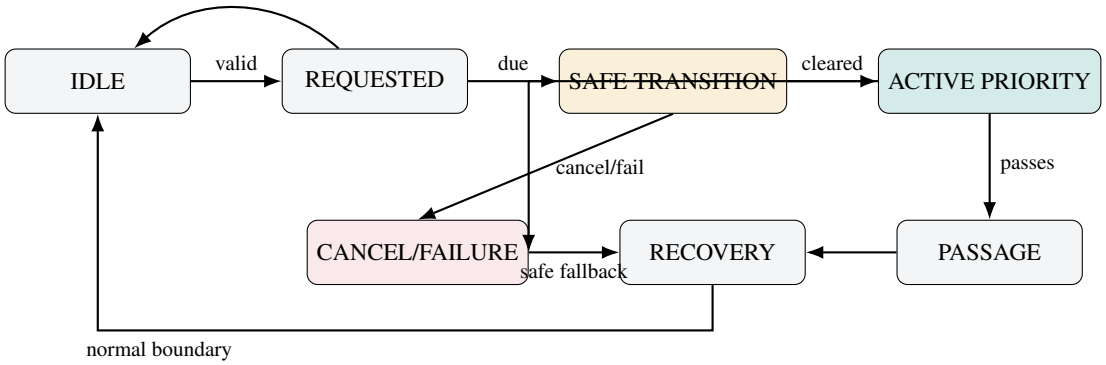


Figure 6.6: Intersection-level moving-corridor state transitions

would be misleading. When activation becomes due, SAFE TRANSITION waits for the current phase's minimum service and completes the configured yellow and all-red clearance. ACTIVE PRIORITY serves only the ambulance movement and is bounded by a maximum duration. Confirmed clearance creates a PASSAGE event, after which RECOVERY restores a valid normal boundary before ordinary control resumes. Cancellation, timeout, or route revision during SAFE TRANSITION or ACTIVE PRIORITY enters the guarded failure and recovery path because signal ownership may already have changed. The distinction prevents an unowned queued request from triggering unnecessary compensation while ensuring that every applied intervention returns safely to IDLE. Each transition records its trigger, route version, and effective step.

6.7.2 Interface and Event Contracts

Table 6.3 lists the minimum fields carried across important module boundaries. Version and time fields prevent a consumer from accepting an unknown schema or treating an old snapshot as current. Source mode, units, confidence, and validity distinguish simulator values from calibrated or proxy video observations. SignalState identifies both the legal successors and current owner, while ControlRequest has no direct side effect and can be transformed into an authoritative AppliedControl record by the safety path. RoutePlan and PriorityRequest carry the mission and route version needed to reject obsolete corridor work. MetricSample retains its definition version and interval so summaries cannot combine measurements whose meaning changed during development. Field names are conceptual and may be mapped to Python or JSON naming conventions, but their semantics remain stable. This contract-first approach also makes integration tests possible before every research component is complete, because a stub can produce valid messages without bypassing ownership rules.

Table 6.3: Core interface and data-contract fields

Contract	Required fields	Rule
TrafficSnapshot	schemaVersion, sourceMode, producedAt, effectiveStep, validUntil, networkId	Consumers reject unknown schema or expired validity; wall time and simulation time are distinct.
ApproachState	intersectionId, approachId, count, queue, wait, speed, occupancy, units, confidence	Missing is not zero; uncalibrated physical quantities are omitted.
SignalState	intersectionId, programId, phaseId, phaseElapsed, legalNext, owner	Owner is NORMAL or CORRIDOR; legal next phases come from validated configuration.
ControlRequest	controllerId, intersectionId, requestedPhase, duration/extension, snapshotStep, reason	Request has no side effect and may be rejected or transformed.
AppliedControl	requestId, appliedPhase, effectiveStep, safetyIntervention, reason	This record is authoritative for analysis.
RoutePlan	missionId, routeVersion, edgeIds, costSeconds, snapshotStep, trigger	Edge order must be continuous from valid current position.
PriorityRequest	missionId, routeVersion, intersectionId, movementId, ETA, expiresAt	Obsolete route version or expired request is rejected.

Contract	Required fields	Rule
MetricSample	runId, metricName, metricVersion, interval, value, unit	Aggregation across mismatched versions is prohibited.

Event ordering is based on simulation step within a run and a monotonic sequence number assigned by the orchestrator. Wall-clock timestamps support diagnostics but do not determine traffic causality because inference, rendering, and storage latency can vary independently of simulated time. A snapshot identifies the step at which its values became effective, and an applied signal event records the step at which the adapter enacted it. Route revisions and priority events are enqueued in an order that prevents an old request from being interpreted as current. Missing values remain missing rather than being converted to zero, and unknown schema versions are rejected rather than partially decoded. These rules give the fields in Table 6.3 operational meaning and support replay, stale-data handling, and comparison across runs without assuming perfectly synchronized wall clocks. They also allow a delayed write to be distinguished from a delayed decision during failure analysis and experimental replay.

6.7.3 Resource Ownership Tactics

The orchestrator is the only writer to simulation time, and TraCI access is serialized behind the adapter. Controller computation receives an immutable snapshot and returns a request rather than holding a reference to mutable global state. Perception uses a bounded work queue; if inference falls behind, the worker marks observations late or drops work according to a recorded policy instead of blocking control indefinitely. Dashboard streams are lossy views, so a slow client may skip intermediate snapshots but cannot delay decisions. Run configuration becomes immutable after start. The current route version is updated atomically with cancellation of obsolete corridor requests, preventing new and old paths from owning priority concurrently. Database writes may be batched where safe, but an applied signal event or mission transition is not acknowledged until it has entered a durable bounded queue. During shutdown, the API stops accepting new commands, the orchestrator ends or detaches SUMO safely, queued records are flushed, and the run receives an explicit completed, stopped, failed, or incomplete terminal state.

6.7.4 Error Handling and Recovery

Table 6.4 defines immediate containment and retained evidence for failures at each major boundary. A SUMO or TraCI failure stops decision-making because continuing to show live state would be misleading, whereas a dashboard disconnect leaves the authoritative experiment unaffected. Stale observations

and invalid controller requests are rejected or handled through a declared simulator or fixed-control fallback, and every interval using that fallback is marked so it cannot be included silently in a clean learned-controller aggregate. A missing route prevents partial corridor activation, while a lost ambulance or timeout cancels future priority and moves active intersections through recovery. Persistence uses a bounded buffer only long enough to tolerate a transient issue; the run stops before evidence loss grows without limit and is marked incomplete if durability cannot be restored. These policies favor an explicit failed run over an apparently successful record whose causal events are missing. Fault-injection and small-network integration tests will verify both the immediate behavior and the final recorded state.

Table 6.4: Failure handling strategy

Failure	Immediate behavior	Recorded/recovery behavior
SUMO launch/TraCI disconnect	Stop decisions and do not present live state	Mark run failed; retain command, stderr, step, and manifest
Stale observation	Exclude or use declared simulator fallback	Record staleness and selected fallback source
Invalid controller action	Reject through safety filter	Apply fixed/safe continuation and record intervention
RL service unavailable	Switch at valid boundary to configured baseline	Record fallback interval; exclude from clean RL aggregate or report separately
No route	Do not spawn/activate partial corridor	Mark mission blocked; show cause and graph snapshot ID
Lost ambulance/timeout	Cancel future priority and recover active intersections	Mark mission failed/cancelled and preserve last position
Persistence failure	Buffer within a bounded queue; stop before unbounded evidence loss	Mark run incomplete if durability cannot be restored
Dashboard disconnect	Continue experiment unaffected	Reconnect from current snapshot and event cursor

6.7.5 Implementation Policies

6.7.5.1 Controller Policy

All controllers implement a common method that accepts an immutable traffic snapshot, intersection configuration, and decision context, then returns a request with explanation fields. They cannot write global state, call TraCI, or suppress the record of a rejected action. The fixed controller, pressure-inspired controller, and learned policy therefore differ in decision logic while sharing phase identifiers, decision intervals, safety constraints, and applied-event recording. Deterministic evaluation disables exploration and seeds every permitted stochastic operation. The safety filter verifies phase membership, minimum green, required clearance, maximum priority duration, and current ownership before an action reaches the adapter. If a policy is unavailable or repeatedly invalid, control transfers at a legal boundary

to the configured baseline and the affected interval is marked. This common interface is central to a fair comparison because it prevents one controller from receiving more permissive movements or direct simulator privileges. Explanation fields are retained for later error analysis.

6.7.5.2 Routing Policy

The graph builder derives nodes and edges from validated SUMO vehicle movements, including allowed classes and turn continuity. Cost providers are composable but must state units, observation time, fallback behavior, and whether link travel time already includes queue delay. This metadata prevents the routing service from adding overlapping estimates or comparing seconds with an unnormalized congestion score. A published route plan is immutable, and the mission points to exactly one current version. Revisions are accepted only through Algorithm 1, which compares paths under one cost snapshot and applies feasibility, hysteresis, persistence, and cooldown rules. A hard closure overrides ordinary cost and invalidates an affected remaining path immediately. Before publication, the route is checked from the ambulance's current edge to the destination and given a revision cause. The routing service never actuates signals; it supplies versioned plans to the corridor coordinator, which can then cancel or create priority requests consistently.

6.7.5.3 Persistence Policy

The store uses schema migrations and referential constraints for structured configuration, events, missions, routes, and metrics, while large videos, model files, and logs may remain checksummed external artifacts with provenance. Raw high-frequency state can be downsampled for persistence only when the sampling interval and aggregation rule are recorded in the metric definition. Completed run configuration is immutable, and derived summaries retain their source run identifiers and analysis version. Secrets, machine-specific credentials, and unportable local paths are excluded from research exports; licensed artifacts are referenced rather than redistributed when necessary. PostgreSQL is proposed for structured records, but implementation may begin with a file-backed engine behind the same repository interface. The domain schema and integrity rules remain independent of that choice so a storage migration cannot change the meaning of an experiment. A persistence failure follows the bounded behavior in Table 6.4 rather than silently discarding events. Recovery records state whether buffered evidence was restored.

6.7.5.4 User-Interface Policy

The user interface is a view and command boundary, not a second control plane. Colors are supplemented with text and symbols so congestion, corridor state, or failure is not conveyed by color alone.

Values display their units, source mode, freshness, and invalid state where applicable. Schematic predictions remain visually distinct from measured results, and an uncalibrated video proxy is not labeled as a physical queue or speed. The interface does not display "time saved" until a valid reference run exists under the same comparison manifest, and it exposes configuration differences when direct comparison is unsafe. Commands receive validation errors tied to fields, while connection loss leaves the experiment running and allows a client to reconnect from the current event cursor. These policies implement the wireframe intentions in Section 4.8 and ensure that a polished display cannot conceal stale data, fallback intervals, missing runs, or incomplete evidence.

6.7.6 Testing Strategy

Unit tests cover cost functions, route continuity, hysteresis, metric calculations, schema validation, freshness rules, and corridor transition guards. Property or invariant tests generate action sequences to ensure incompatible greens are never requested for application, one component owns each intersection, obsolete route versions cannot activate priority, and cancellation eventually reaches IDLE through RECOVERY. Integration tests use small SUMO networks to validate single-step ownership, detector-to-state messages, route application, fallback, and persistence ordering. Scenario tests replay fixed seeds for normal control, closure rerouting, no-route, invalid actions, stale data, controller loss, ambulance timeout, persistence failure, and dashboard disconnect. Contract fixtures allow components to be replaced by stubs without weakening field validation. Research verification remains separate from software correctness: passing these tests shows that the implementation follows its specification, but it does not establish that a policy improves traffic or emergency travel. The matched experiments in Chapter 5 provide that evidence, while test results explain whether a failed experiment reflects the policy or a software defect.

6.7.7 Coding, Build, and Maintainability Policies

Python modules will use static type hints, consistent formatting and linting, bounded dependency versions, structured logging, and documentation for public interfaces. React components separate data acquisition from presentation so a visual revision cannot change experiment ownership. Configuration is stored in validated versioned files rather than scattered constants, and secrets remain outside exported manifests. Continuous checks should run unit tests, schema validation, and a minimal headless SUMO scenario before a revision is used for evidence. Experiment releases are tagged and their commit identifier is written into each run manifest. The intended repository structure separates application services, controllers, routing, perception, simulation, dashboard, contracts, scenarios, and experiment artifacts. This is a proposed organization, not evidence that all directories or services already exist. Small interfaces allow a tracker, learned policy, database engine, or dashboard transport to change without rewriting

the central loop, while versioned contracts force such changes to declare any effect on recorded evidence.

6.7.8 Continuous Integration and Deployment

The proposed continuous-integration pipeline runs formatting, linting, type checks, unit tests, contract fixtures, schema validation, and a minimal headless SUMO scenario. A build that changes an interface or metric definition must update its version and associated fixtures. Deployment for the Deliverable 2 prototype remains a documented local environment rather than a claimed production service. A tagged demonstration build includes dependency locks, validated scenarios, required model checksums, and migration state so another team member can reproduce the same prepared workflow. The headless smoke scenario verifies process launch, one controlled step sequence, valid signal application, event persistence, and a clean terminal state without requiring the dashboard. Research experiments are not automatically promoted merely because software tests pass; the matched evaluation protocol remains a separate activity. Failed checks block an evidence release, and bypasses must be recorded rather than hidden. If an external model or licensed video cannot be packaged, the build manifest records its checksum and acquisition instructions while keeping the protected artifact outside the archive.

6.7.9 Source Code Organization

The intended repository separates API and application services from controllers, routing, perception, simulation adapters, interface contracts, dashboard code, scenarios, and experiment artifacts. Tests mirror those boundaries, while generated outputs and large external artifacts remain outside source modules. Configuration is versioned and validated instead of embedded in code. This arrangement keeps research implementations replaceable, makes ownership visible during review, and avoids circular dependencies between interface, simulation, and persistence concerns. Domain objects and contracts occupy a low-dependency package that does not import the dashboard or infrastructure adapters. Controller implementations share an interface but do not import each other, allowing fixed, pressure-inspired, and learned policies to be selected by configuration. Scenario manifests reference networks and demand files through portable paths and checksums. Analysis scripts read retained records rather than private controller state. Documentation accompanies public interfaces and configuration schemas. This organization remains proposed until source implementation confirms it, but it defines a reviewable target that aligns folders, dependency direction, component ownership, and the architectural diagrams in this chapter.

6.8 Conclusion

The design assigns one clear owner to every mutable resource and makes observation source, simulation time, units, route version, requested action, and applied action explicit. The high-level architecture keeps presentation and perception outside the single-writer simulation path, while component responsibilities and data contracts make integration boundaries testable. The domain model and sequences preserve causal links from scenario and snapshot through controller request, safety decision, route revision, corridor transition, and durable event. Guarded recovery and bounded fallbacks ensure that a failure does not leave an unexplained signal or apparently live dashboard. Common controller interfaces support the intended progression from fixed and pressure-inspired baselines to a learned policy, and versioned routing enables a moving corridor without granting the pathfinder direct signal authority. The architecture is modular enough for team implementation on one machine and leaves distribution as a profiling decision, not an assumption. It also preserves failed runs and negative research outcomes as evidence instead of hiding them behind interface behavior.

Bibliography

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[2] United Nations, "Goal 11: Sustainable cities and communities." https://sdgs.un.org/goals/goal11, 2025. Accessed: 6 October 2026.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 779–788, 2016.

[4] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *2017 IEEE International Conference on Image Processing*, pp. 3645–3649, 2017.

[5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "Bytetrack: Multi-object tracking by associating every detection box," in *European Conference on Computer Vision*, pp. 1–21, 2022.

[6] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving dataset for heterogeneous multitask learning," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2636–2645, 2020.

[7] L. Wen, D. Du, Z. Cai, Z. Lei, M.-C. Chang, H. Qi, J. Lim, M.-H. Yang, and S. Lyu, "Ua-detrac: A new benchmark and protocol for multi-object detection and tracking," *Computer Vision and Image Understanding*, vol. 193, p. 102907, 2020.

[8] P. Varaiya, "Max pressure control of a network of signalized intersections," *Transportation Research Part C: Emerging Technologies*, vol. 36, pp. 177–195, 2013.

[9] X. Liang, X. Du, G. Wang, and Z. Han, "A deep reinforcement learning network for traffic light cycle control," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 2, pp. 1243–1253, 2019.

[10] H. Wei, C. Chen, G. Zheng, K. Wu, V. Gayah, K. Xu, and Z. Li, "Presslight: Learning max pressure control to coordinate traffic signals in arterial network," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1290–1298, 2019.

[11] G. Zheng, X. Zang, N. Xu, H. Wei, Z. Yu, V. Gayah, K. Xu, and Z. Li, "Learning phase competition for traffic signal control," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pp. 1963–1972, 2019.

[12] H. Wei, N. Xu, H. Zhang, G. Zheng, X. Zang, C. Chen, W. Zhang, Y. Zhu, K. Xu, and Z. Li, "Colight: Learning network-level cooperation for traffic signal control," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pp. 1913–1922, 2019.

[13] T. Chu, J. Wang, L. Codecà, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1086–1095, 2020.

[14] H. Zhang, S. Feng, C. Liu, Y. Ding, Y. Zhu, Z. Zhou, W. Zhang, Y. Yu, H. Jin, and Z. Li, "Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario," in *The World Wide Web Conference*, pp. 3620–3624, 2019.

[15] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using sumo," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pp. 2575–2582, 2018.

[16] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[17] S. Humagain, R. Sinha, E. Lai, and P. Ranjitkar, "A systematic review of route optimisation and pre-emption methods for emergency vehicles," *Transport Reviews*, vol. 40, no. 1, pp. 35–53, 2020.

[18] X. Qin and A. M. Khan, "Control strategies of traffic signal timing transition for emergency vehicle preemption," *Transportation Research Part C: Emerging Technologies*, vol. 25, pp. 1–17, 2012.

[19] A. Louati, S. Elkosantini, S. Darmoul, and H. Louati, "Multi-agent preemptive longest queue first system to manage the crossing of emergency vehicles at interrupted intersections," *European Transport Research Review*, vol. 10, no. 2, p. 52, 2018.

[20] Eclipse Foundation, "Sumo documentation: Traffic lights and traci." https://sumo.dlr.de/docs/, 2026. Accessed: 6 October 2026.

[21] Meta Open Source, “React documentation.” https://react.dev/, 2026. Accessed: 6 October 2026.

[22] FastAPI, “Fastapi documentation.” https://fastapi.tiangolo.com/, 2026. Accessed: 6 October 2026.

[23] OpenStreetMap Contributors, “Openstreetmap copyright and license.” https://www.openstreetmap.org/copyright, 2026. Accessed: 6 October 2026.